Fayoum Racing Team

Autonomous Technical Team

ROS2 Humble

Comprehensive Documentation & Guide

Prepared by:

Ammar Yasser

President of Fayoum Racing Team

Contents

Strategic Roadmap: The 7 Phases of the Fayoum Racer

1. Phase 1: The Nervous System (Core Communication)

> *"To build the fundamental 'Nervous System' of the racer. This phase isn't just about code; it's about establishing how every sensor and motor acts as a single, synchronized organism using the ROS 2 Computation Graph."*

What the team will master:

- Transforming a single script into a distributed network of **Nodes**.
- Mastering the **Executor** logic to ensure no sensor data is lost during high-speed processing.
- Adhering to the **ROS 2 SOP** to maintain professional-grade code stability.

—

2. Phase 2: Integrated Communication Protocols

> *"To master the language of robotics. We are defining the protocols for how the car 'talks'—from the non-stop stream of LiDAR data to the critical mission commands that must never fail."*

What the team will master:

- Creating robust **Publishers/Subscribers** for continuous telemetry and sensor feeds.
- Designing **Services** for instant, reliable hardware triggers (like resetting an encoder).
- Implementing **Actions** for complex maneuvers with proactive cancellation safety.

—

3. Phase 3: The Command Center (Orchestration & Tools)

> *"To take full control of the vehicle's behavior. This is where we move from running parts to orchestrating a complete racing machine that can be tuned, launched, and diagnosed with professional precision."*

What the team will master:

- Tuning performance in real-time using **Dynamic Parameters** without restarting the system.
- Automating the entire autonomous stack deployment with a single **Master Launch File**.
- Utilizing **Rosbag2** as our 'Flight Data Recorder' and **RViz2** as our visual eyes.

—

4. Phase 4: Spatial Awareness & Kinematics

> *"To give the racer a sense of space. We are building the 'Digital Geometry' of the car, ensuring that every sensor knows exactly where it is relative to the track and the chassis, down to the last millimeter."*

What the team will master:

- Managing complex **Coordinate Frames** (Map, Odom, Base_link) to prevent localization errors.
- Solving real-world racing problems like sensor displacement and wheel-to-lidar calibration.
- Mastering **TF2 Listeners and Broadcasters** to maintain a perfect 3D transform tree.

—

5. Phase 5: Advanced Node Architecture

> *"To optimize the racer for extreme performance. We are moving beyond standard nodes to build a high-speed, deterministic system that can handle 4K vision and high-freq LiDAR without breaking a sweat."*

What the team will master:

- Using **Composition** and **Zero-Copy** communication to eliminate CPU bottlenecks.
- Enforcing a **Deterministic Boot Sequence** using **Lifecycle Nodes** for safety.
- Orchestrating complex shared-memory containers for vision-heavy perception tasks.

—

6. Phase 6: Hardware & Simulation Integration

> *"To bridge the gap between code and reality. We are creating the 'Digital Twin' of the Fayoum Racer, allowing us to test dangerous maneuvers in Gazebo before ever touching the physical track."*

What the team will master:

- Describing the car's physics and geometry using modular **URDF/Xacro** files.
- Implementing **ros2_control** to handle the 'Muscles' (Ackermann steering and motor torque).
- Mastering the **Gazebo Physics Bridge** to simulate real-world sensor feeds.

—

7. Phase 7: The Autonomous Stack

> *"To awaken the Master Driver. This is the ultimate goal: fusing sensors, mapping the track, and planning the optimal racing line to win the race autonomously."*

What the team will master:

- Implementing the **Extended Kalman Filter (EKF)** to find the "Filtered Truth."
- Building high-resolution track maps using **SLAM** while localizing at high speeds.
- Orchestrating the **Nav2 Framework** to compute the fastest path and avoid obstacles.

CHAPTER 1

Phase 1: The Nervous System (Core Communication)

1.1 The Core Concept: What is a "Robotics Middleware"?

The "Why": Why can't we just write a standalone C++ program?

Imagine you want to build an autonomous race car. Your first instinct might be to open an IDE and write a massive C++ program with a giant `while(true)` loop: read the LiDAR, process the camera frames, calculate the path, and send PWM signals to the steering servo.

This monolithic approach fails instantly in the real world for three reasons:

- **Concurrency and Bottlenecks:** Processing a 4K camera stream takes significant compute time. If your `while` loop is stuck waiting for an image frame, your car isn't sending braking commands. You will crash.
- **The Single Point of Failure:** In a monolithic script, a segmentation fault caused by a buggy camera driver crashes the entire program. The steering and brakes freeze because the camera failed.
- **Reinventing the Wheel:** You would have to write custom drivers for every single piece of hardware from scratch, parsing raw bytes over USB or Ethernet.

The "How": The Middleware Solution

ROS is a "Middleware." It sits between the operating system (like Linux) and your specific robotics software. It provides a standardized framework that allows you to break that giant monolithic C++ program into dozens of small, independent, specialized programs (called **Nodes**) that run concurrently and talk to each other.

1.2 The Philosophy: The Microservices / Nervous System Analogy

Think of ROS like a biological nervous system or a modern cloud microservices architecture.

In the human body, your eyes don't know how your legs work. Your eyes simply capture light, convert it into standard neural signals, and publish them to the brain. The brain subscribes to those signals, makes a decision, and publishes movement commands to the legs.

ROS does exactly this for robots:

- **A Camera Node** (the eye) only knows how to talk to a specific physical camera. It translates raw USB data into a standard ROS `Image` message and broadcasts it.
- **A Perception Node** (the visual cortex) subscribes to that `Image`, detects track boundaries, and broadcasts a `TrackBounds` message.
- **A Control Node** (the motor cortex) subscribes to the bounds and calculates steering angles.

If you swap your camera for a completely different brand, the Perception and Control nodes don't care. They don't need to be rewritten. They just keep waiting for that standard `Image` message.

1.3 The 4 Pillars of ROS

To make this distributed nervous system work, ROS provides four fundamental pillars:

- **1. Hardware Abstraction (The Drivers):**
  What it is: ROS provides pre-written, open-source packages that act as translators between raw hardware and the ROS ecosystem.
  Real-world Use Case: When you buy a Hokuyo LiDAR or an Ouster 3D LiDAR, you don't read the manufacturer's manual to decode byte streams. You just run the ROS driver provided by the manufacturer or community, and instantly, standard `LaserScan` or `PointCloud2` messages start flowing into your system.
- **2. Message Passing (The Plumbing):**
  What it is: The standardized communication protocols (which we will cover via DDS) that allow a Python script running your AI to seamlessly pass data to a highly optimized C++ node handling motor control, as if they were the same program.
  Real-world Use Case: A complex autonomous vehicle might have an object detection algorithm written rapidly in Python, streaming bounding box coordinates to a path-planning algorithm written in Modern C++ 17 for maximum performance. ROS handles the serialization and network transit between the two languages invisibly.
- **3. Package Management (The Ecosystem):**
  What it is: A standardized way to organize, build, and share code.
  Real-world Use Case: You don't need to write a complex navigation algorithm from scratch. Because ROS uses standardized packages, you can download the `Nav2`

package, configure it for your vehicle's kinematics, and instantly have state-of-the-art autonomous navigation.

- **4. Tools (The Laboratory):**
  What it is: A suite of powerful GUI tools for visualization, debugging, and simulation.
  Real-world Use Case: Before putting a physical vehicle on a track—where a software bug means destroyed hardware—you use Gazebo to simulate the physics of the car and the sensors. You use RViz2 to visually inspect what the robot's "brain" is seeing in real-time (e.g., overlaying the LiDAR point cloud onto a 3D grid).

Best Practices & Common Pitfalls

- **Pitfall – The Monolithic Node:** Beginners often adopt ROS but still write "God Nodes"—a single ROS node that handles camera processing, path planning, and control all at once. This defeats the entire purpose of the framework.
- **Best Practice – Single Responsibility Principle:** A node should do exactly one thing. One node reads the sensor. One node filters the data. One node plans the path. This makes debugging easy: if the path is bad, you check the output of the filtering node. If the filter output is good, you know the bug is strictly isolated inside the planner.

1.4 The Computation Graph (Nodes and Executors)

1.4.1 Part 1: The Node (The Fundamental Worker)

The "Why"

As we discussed, a monolithic program is a liability. We need a way to encapsulate specific hardware interactions or algorithms into isolated, independent processes. The "Node" is the atomic unit of computation in ROS 2. It is the entity that connects to the DDS middleware, registers itself on the network, and handles its own specific job.

The "How" (Architecture Level)

Under the hood, a ROS 2 Node is not a standalone magical entity; it is simply a C++ or Python class that inherits from the ROS Client Library base classes (`rclcpp::Node` or `rclpy.node.Node`). When you instantiate this class, it communicates with the underlying DDS implementation to announce its existence to the rest of the computation graph.

Use Cases

- **The Sensor Driver:** A node specifically dedicated to reading wheel encoders on an autonomous race car to calculate raw speed.
- **The Safety Watchdog:** A lightweight node that constantly monitors system diagnostics and triggers an emergency stop if the main planner crashes.

Implementation: Writing Your First Node

Industry standard dictates that ROS 2 code must always be Object-Oriented. We never write procedural scripts in the global scope. We will write a simple "Telemetry Node" that boots up and logs its status at a fixed frequency.

C++ Implementation (Modern C++ 14/17)

Create this in `src/telemetry_node.cpp`:

```cpp
#include <chrono>               // Standard library for time/duration
#include <memory>               // Standard library for smart pointers (Modern C++)
#include "rclcpp/rclcpp.hpp"    // The core ROS 2 C++ client library
using namespace std::chrono_literals; // Allows us to write '500 ms' instead of verbose chrono calls

// 1. Inherit from the base rclcpp::Node class
class TelemetryNode : public rclcpp::Node
{
public:
    // 2. The Constructor: Name the node "telemetry_monitor" on the ROS graph
    TelemetryNode() : Node("telemetry_monitor"), count_(0)
    {
        // 3. Create a timer that fires every 500 milliseconds.
        // std::bind hooks the timer to our member function 'timer_callback'
        timer_ = this->create_wall_timer(
            500ms, std::bind(&TelemetryNode::timer_callback, this));

        // Log a startup message to the console
        RCLCPP_INFO(this->get_logger(), "Telemetry Node Initialized. Systems nominal.");
    }
private:
    // 4. The Callback: This function executes every time the timer fires
    void timer_callback()
    {
        RCLCPP_INFO(this->get_logger(), "Publishing telemetry packet #%d", count_++);
    }

    // Member variables
    rclcpp::TimerBase::SharedPtr timer_; // Smart pointer managing the timer's lifecycle
    size_t count_;                       // Simple counter
};

int main(int argc, char * argv[]) {
```

```cpp
    // A. Initialize the ROS 2 communications network
    rclcpp::init(argc, argv);

    // B. Instantiate our node using a shared pointer (Modern C++ best practice)
    auto node = std::make_shared<TelemetryNode>();

    // C. Hand the node over to the Executor to spin (block and process callbacks)
    rclcpp::spin(node);

    // D. Clean shutdown of the ROS 2 network when the node is killed (Ctrl+C)
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

Create this in `telemetry_node.py`:

```python
import rclpy                                # The core ROS 2 Python client library
from rclpy.node import Node                 # The base Node class

# 1. Inherit from the base rclpy.node.Node class
class TelemetryNode(Node):

    def __init__(self):
        # 2. The Constructor: Name the node "telemetry_monitor" on the ROS graph
        super().__init__('telemetry_monitor')
        self.count_ = 0

        # 3. Create a timer that fires every 0.5 seconds, triggering 'timer_callback'
        self.timer_ = self.create_timer(0.5, self.timer_callback)

        # Log a startup message to the console
        self.get_logger().info('Telemetry Node Initialized. Systems nominal.')

    # 4. The Callback: This function executes every time the timer fires
    def timer_callback(self):
        self.get_logger().info(f'Publishing telemetry packet #{self.count_}')
        self.count_ += 1

def main(args=None):
    # A. Initialize the ROS 2 communications network
    rclpy.init(args=args)
```

```
26
27      # B. Instantiate our node
28      node = TelemetryNode()
29
30      # C. Hand the node over to the Executor to spin (block and
            process callbacks)
31      rclpy.spin(node)
32
33      # D. Clean shutdown of the ROS 2 network when the node is
            killed (Ctrl+C)
34      node.destroy_node()
35      rclpy.shutdown()
36
37  if __name__ == '__main__':
38      main()
```

1.4.2 Part 2: Executors (The Engine of the Node)

The "Why"
Look at the `main` function in both examples. You created a node, but then you called `spin()`. Why? Because a Node by itself is just a passive data structure. It does nothing until an "Executor" gives it a thread to run on. If a network packet arrives from the LiDAR, something has to actively wake up, read the packet, and trigger your callback function.

The "How" (Architecture Level)
The Executor is the actual engine of ROS 2 computation. It manages one or more threads and a queue of callbacks (timers, incoming messages, service requests).

`rclcpp::spin(node)` creates a Single-Threaded Executor by default. It takes the calling thread (the main thread) and locks it into an infinite loop, checking the DDS queue and executing callbacks one by one, in order.

Use Cases

- **Single-Threaded Executor:** Perfect for our Telemetry Node above. It only has one timer, so one thread is plenty.
- **Multi-Threaded Executor:** Crucial for complex nodes like an autonomous path planner. If the planner receives a massive PointCloud message that takes 100ms to process, you do not want the node to ignore an emergency stop command during those 100ms. A multi-threaded executor allows the emergency command to be processed on thread #2 while thread #1 is busy with the PointCloud.

Best Practices & Common Pitfalls

The Deadliest Sin: Blocking the Executor Thread.

- **Pitfall:** In a Single-Threaded Executor, if you put `sleep(5)` or a heavy `while(true)` mathematical calculation inside your `timer_callback`, the en-

tire node freezes. No other timers will fire. No incoming messages will be read. Your vehicle will fly completely blind for 5 seconds.
- **Best Practice:** Callbacks must be lightning-fast. They should read data, update state variables, and exit immediately. If you have heavy computation, push it to a separate background thread or use a Multi-Threaded Executor with mutually exclusive Callback Groups.

Object-Oriented Design (OOD):
- **Pitfall:** Declaring global variables to share data between different ROS callbacks.
- **Best Practice:** As demonstrated in the code, encapsulate everything within the class state (e.g., `self.count_`). This guarantees thread safety (when configured properly) and allows you to instantiate multiple instances of the same node type in a single process later on.

1.4.3 Part 3: The ROS 2 Development Lifecycle (Standard Operating Procedure)

The "Why"
Without a strict SOP, engineers will compile code in the wrong directories, overwrite environment variables, or run outdated binaries. This lifecycle enforces the "Underlay/Overlay" architecture we discussed earlier, ensuring that your custom code seamlessly sits on top of the base ROS 2 framework without corrupting it.

The "How" (The Rhythm)
This is the chronological loop you will execute hundreds of times a week as a ROS 2 developer.

Step 1: Environment Setup (The Underlay)
Before you can use any ROS 2 tools (`ros2`, `colcon`), your terminal needs to know they exist. You must source the base ROS 2 installation.

Terminal
```
# This injects ROS 2 Humble commands and libraries into your current
terminal.
source /opt/ros/humble/setup.bash
```

Step 2: Workspace Creation
You isolate your project from the rest of your Linux system by creating a dedicated workspace with a `src` directory.

Terminal
```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws/src
```

Step 3: Package Generation
You generate the boilerplate architecture for your specific module.

Terminal
```
# For a C++ Node:
ros2 pkg create my_cpp_pkg --build-type ament_cmake --dependencies
rclcpp

# OR For a Python Node:
ros2 pkg create my_python_pkg --build-type ament_python
--dependencies rclpy
```

Step 4: Implementation (Code & Build Registration)
This is where you write the actual logic (the `telemetry_node` code we wrote in Part 1).
Crucial Step: You must register your node so the build system knows it exists.

- **C++:** You edit `CMakeLists.txt` to add the executable (`add_executable`) and

install rules.

- **Python:** You edit `setup.py` and add your node to the `console_scripts` entry points.

Step 5: Compilation (colcon)
You return to the root of your workspace to compile. You **never** compile from inside the `src` folder.

Terminal

```
cd ~/ros2_ws

# The --symlink-install flag is mandatory for modern development.
# It creates symbolic links to Python scripts and config files.
# This means if you edit a Python file, you do NOT have to rebuild!
colcon build --symlink-install
```

Step 6: Environment Registration (The Overlay) - CRITICAL
`colcon` just built your code and put the binaries in the `install` folder. However, your terminal still does not know they exist. You must source the newly created overlay.

Terminal

```
# This tells your terminal: "Look in here for my custom packages."
source install/setup.bash
```

Step 7: Execution
Now, and only now, can you run the node.

Terminal

```
ros2 run my_cpp_pkg telemetry_node
```

Step 8: Verification
To verify it works, you must open a new terminal. Because it is a new terminal, it is completely blank. You must repeat the sourcing steps before using the CLI tools.

Terminal

```
# In Terminal 2:
source /opt/ros/humble/setup.bash
source ~/ros2_ws/install/setup.bash

# Interrogate the graph
ros2 node list
ros2 node info /telemetry_monitor
```

Best Practices & Common Pitfalls

- **Pitfall – The "Ghost Code" Phenomenon:** You edit a C++ file, type

`ros2 run`, and nothing changes. You edit it again, still nothing. Why? Because you forgot to run `colcon build`. C++ must always be recompiled.

- **Pitfall – The "Cross-Contaminated Workspace":** If you have multiple workspaces (e.g., `navigation_ws` and `perception_ws`), never source them both in the same terminal while building. Sourcing an overlay before running `colcon build` can accidentally link your new code to the wrong dependencies.
- **Best Practice – The `.bashrc` Automation:** Typing `source /opt/ros/humble/setup.bash` in every single new terminal is exhausting. Engineers automate this by adding it to their `~/.bashrc` file so it runs automatically every time a terminal opens. However, you generally do **not** automate sourcing your custom overlay (`install/setup.bash`) to prevent cross-contamination during active development.

1.4.4 Part 4: Inspecting the Computation Graph (CLI Tools)

The "Why"
ROS 2 runs as a distributed system. You might have 50 different nodes running simultaneously across three different computers on the same network. We need specialized Command Line Interface (CLI) tools to launch these nodes, verify their existence, and dissect their internal connections in real-time without stopping the system.

The "How" (Architecture Level)
Under the hood, these CLI tools are essentially temporary, hidden ROS 2 nodes themselves. When you type a command like `ros2 node list`, the CLI spins up a temporary node, queries the DDS discovery protocol for active participants, prints the results to your terminal, and immediately shuts itself down.

1. Executing the Code: `ros2 run`
The "Why": You cannot simply run a ROS 2 executable by typing `./telemetry_node`. The executable needs the ROS 2 environment variables, workspace paths, and middleware configurations to successfully join the network.
The "How": `ros2 run` searches your currently sourced workspaces (both the `/opt/ros/humble` underlay and your custom `ros2_ws` overlay) to locate the package and its registered executables.

Use Cases

- Booting up an individual hardware driver (e.g., spinning up just the LiDAR node to test if the laser is spinning).
- Testing a newly compiled C++ or Python node in isolation before integrating it into the full software stack.

Implementation

Terminal

```
# Syntax: ros2 run <package_name> <executable_name>

# Example (Assuming we built the C++ package from Part 1):
ros2 run my_cpp_pkg telemetry_node

# Expected Terminal Output:
# [INFO] [1708214532.123456] [telemetry_monitor]: Telemetry Node
Initialized. Systems nominal.
# [INFO] [1708214532.623456] [telemetry_monitor]: Publishing
telemetry packet #0
# [INFO] [1708214533.123456] [telemetry_monitor]: Publishing
telemetry packet #1
```

2. Network Verification: `ros2 node list`
The "Why": Just because a node hasn't crashed in the terminal doesn't mean it successfully connected to the ROS 2 network. We need a way to confirm that the middleware sees the node.
The "How": This command asks the DDS middleware to return the namespace and node name of every active participant it currently tracks.

Use Cases

- **Sanity Checks:** "I just started the camera driver, but the perception algorithm is failing. Is the camera node even alive on the network?"
- **Network Partition Debugging:** If you run this on your laptop and don't see the nodes running on the robot's onboard computer, you instantly know you have a Wi-Fi or DDS configuration issue, not a code bug.

Implementation
Open a new terminal (ensure you have sourced your workspace) while your `telemetry_node` is running:

Terminal

```
ros2 node list

# Expected Terminal Output:
# /telemetry_monitor
```

3. Deep Introspection: `ros2 node info`
The "Why": Knowing a node is alive is only step one. We need to know its interface: What data is it broadcasting? What data is it listening for? What services does it offer? If node A isn't talking to node B, `node info` is your diagnostic scalpel.
The "How": This command queries the specific node directly over the ROS 2 graph to map out its active Publishers, Subscribers, Service Servers, Service Clients, and Action Servers.

Use Cases

- **Debugging Communication Drops:** Verifying if the motor control node is actually subscribed to the `/cmd_vel` steering topic, or if there is a typo in the code.
- **Exploring Third-Party Code:** When you download a community package (like an IMU driver) and want to quickly see what topics it outputs without digging through its source code.

Implementation

Terminal

```
# Syntax: ros2 node info <node_name>
ros2 node info /telemetry_monitor

# Expected Terminal Output:
# /telemetry_monitor
#   Subscribers:
#     /parameter_events: rcl_interfaces/msg/ParameterEvent
#   Publishers:
#     /parameter_events: rcl_interfaces/msg/ParameterEvent
#     /rosout: rcl_interfaces/msg/Log
#   Service Servers:
#     /telemetry_monitor/describe_parameters:
rcl_interfaces/srv/DescribeParameters
#     /telemetry_monitor/get_parameters: rcl_interfaces/srv/GetParamete
#     ...
```

Note: Even though we didn't explicitly write any publishers or subscribers in our C++ code yet, ROS 2 automatically attaches default parameter and logging interfaces (`/rosout`) to every node.

Best Practices & Common Pitfalls

- **Pitfall – Relying on `ros2 run` for Production:** Beginners try to start an entire robot by opening 15 terminals and typing `ros2 run` in each one. This is unmanageable.
- **Best Practice:** Use `ros2 run` strictly for isolated development and debugging. When deploying the full system, you will use `ros2 launch` (which we will cover in Module 3.2) to bring up dozens of nodes deterministically with a single command.
- **Pitfall – Name Collisions:** If you use `ros2 run` to start the exact same node twice, the second node will boot up with the identical name (`/telemetry_monitor`). This causes massive routing confusion in the DDS middleware. Always ensure node names are unique, or use namespaces (e.g., `/robot1/telemetry_monitor`).

CHAPTER 2

Phase 2: Integrated Communication Protocols

2.1 Module 2.1: Topics (Continuous Data Streams)

2.1.1 Part 1: Topics (Continuous Data Streams)

The "Why"
Imagine an autonomous car driving at 30 km/h. The LiDAR is spinning at 10Hz, generating thousands of data points per second. The motor controller needs constant updates on the target velocity.

You cannot use a "Request/Response" system for this. The motor controller shouldn't have to constantly ask, "What is my speed now? How about now?" Topics exist to solve the need for **continuous, decoupled, unidirectional data streams**.

The "How" (Architecture Level)
Under the hood, Topics use the DDS Publish/Subscribe pattern.

- **Decoupled & Anonymous:** A node publishing to a topic has *no idea* who is listening. A node subscribing to a topic has *no idea* who is sending the data. They only care about the Topic Name (e.g., `/vehicle/speed`) and the Message Type (e.g., `std_msgs/Float32`).
- **Many-to-Many:** One LiDAR node can publish to `/scan`. Five different nodes (Mapping, Obstacle Avoidance, Visualizer, Data Logger, Safety Watchdog) can all subscribe to `/scan` simultaneously without bottlenecking the publisher.

Real-world Autonomous Racing Use Cases

- **Publishers:** Wheel encoders publishing the current physical velocity of the car; a camera driver continuously publishing raw image frames; an IMU publishing linear acceleration and angular velocity.
- **Subscribers:** The dashboard UI listening to the velocity to display a speedometer; the perception algorithm subscribing to the camera frames to detect track bounds;

the Extended Kalman Filter (EKF) subscribing to the IMU to calculate exact vehicle pose.

2.1.2 Part 2: Implementation - The Publisher

We will build a `SpeedPublisher` that simulates reading a wheel encoder and broadcasting the vehicle's speed at 10Hz. We will use a standard ROS 2 message type: `std_msgs/msg/Float32`.

Modern C++ 14/17 Implementation

File: src/speed_publisher.cpp

```cpp
#include <chrono>
#include <memory>
// Core ROS 2 C++ API
#include "rclcpp/rclcpp.hpp"
// The specific message type we are broadcasting
#include "std_msgs/msg/float32.hpp"

using namespace std::chrono_literals;

class SpeedPublisher : public rclcpp::Node
{
public:
    SpeedPublisher() : Node("speed_sensor_node"), current_speed_(0.0)
    {
        // 1. Create the Publisher.
        // Syntax: create_publisher<MessageType>("topic_name", qos_history_depth)
        publisher_ = this->create_publisher<std_msgs::msg::Float32>("/vehicle/speed", 10);

        // 2. Create a timer to fire at 10Hz (100ms) to trigger the publish loop
        timer_ = this->create_wall_timer(
            100ms, std::bind(&SpeedPublisher::timer_callback, this));

        RCLCPP_INFO(this->get_logger(), "Speed Publisher initialized on /vehicle/speed");
    }

private:
    void timer_callback()
    {
        // 3. Instantiate the message object
        auto message = std_msgs::msg::Float32();
        message.data = current_speed_;

        // 4. Publish the message to the DDS network
```

```
34          publisher_->publish(message);
35
36          // Simulate slight acceleration for our test
37          current_speed_ += 0.5;
38      }
39
40      // Class members
41      rclcpp::TimerBase::SharedPtr timer_;
42      rclcpp::Publisher<std_msgs::msg::Float32>::SharedPtr
            publisher_;
43      float current_speed_;
44  };
45
46  int main(int argc, char * argv[])
47  {
48      rclcpp::init(argc, argv);
49      rclcpp::spin(std::make_shared<SpeedPublisher>());
50      rclcpp::shutdown();
51      return 0;
52  }
```

Python Implementation

File: speed_publisher.py

```
1  import rclpy
2  from rclpy.node import Node
3  # The specific message type we are broadcasting
4  from std_msgs.msg import Float32
5
6  class SpeedPublisher(Node):
7      def __init__(self):
8          super().__init__('speed_sensor_node')
9          self.current_speed_ = 0.0
10
11         # 1. Create the Publisher.
12         # Syntax: create_publisher(MessageType, 'topic_name',
               qos_history_depth)
13         self.publisher_ = self.create_publisher(Float32, '/
               vehicle/speed', 10)
14
15         # 2. Create a timer to fire at 10Hz (0.1 seconds)
16         self.timer = self.create_timer(0.1, self.timer_callback)
17         self.get_logger().info('Speed Publisher initialized on /
               vehicle/speed')
18
19     def timer_callback(self):
20         # 3. Instantiate the message object
21         msg = Float32()
22         msg.data = self.current_speed_
23
24         # 4. Publish the message
```

```
        self.publisher_.publish(msg)

        self.current_speed_ += 0.5

def main(args=None):
    rclpy.init(args=args)
    node = SpeedPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2.1.3 Part 3: Implementation - The Subscriber

Now we build a `DashboardSubscriber` that actively listens to `/vehicle/speed` and reacts every time a new packet arrives.

Modern C++ 14/17 Implementation

File: src/dashboard_subscriber.cpp

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/float32.hpp"

// Required for the std::bind placeholder _1
using std::placeholders::_1;

class DashboardSubscriber : public rclcpp::Node
{
public:
    DashboardSubscriber() : Node("dashboard_node")
    {
        // 1. Create the Subscriber.
        // Syntax: create_subscription<MessageType>("topic_name",
            qos_depth, callback_function)
        subscription_ = this->create_subscription<std_msgs::msg::
           Float32>(
            "/vehicle/speed", 10, std::bind(&DashboardSubscriber
               ::speed_callback, this, _1));

        RCLCPP_INFO(this->get_logger(), "Dashboard Subscriber
           listening to /vehicle/speed");
    }

private:
    // 2. The Callback Function. Triggered automatically by the
       Executor when data arrives.
    // It takes a Shared Pointer to the incoming message to
       prevent unnecessary memory copying.
```

```
    void speed_callback(const std_msgs::msg::Float32::SharedPtr
        msg) const
    {
        RCLCPP_INFO(this->get_logger(), "Dashboard reading: %.2f
            m/s", msg->data);
    }

    rclcpp::Subscription<std_msgs::msg::Float32>::SharedPtr
        subscription_;
};

int main(int argc, char * argv[])
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<DashboardSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

File: ***dashboard_subscriber.py***

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Float32

class DashboardSubscriber(Node):
    def __init__(self):
        super().__init__('dashboard_node')

        # 1. Create the Subscriber.
        self.subscription = self.create_subscription(
            Float32,
            '/vehicle/speed',
            self.speed_callback,
            10)
        self.subscription  # Prevent unused variable warning
        self.get_logger().info('Dashboard Subscriber listening to
            /vehicle/speed')

    # 2. The Callback Function. Note the 'msg' parameter.
    def speed_callback(self, msg):
        self.get_logger().info(f'Dashboard reading: {msg.data:.2f
            } m/s')

def main(args=None):
    rclpy.init(args=args)
    node = DashboardSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

```
28
29 if __name__ == '__main__':
30     main()
```

Best Practices & Common Pitfalls

- **Pitfall – The Silent Topic Mismatch:** If your C++ publisher publishes to `/vehicle/speed` but your Python subscriber is accidentally listening to `/Vehicle/Speed` (capitalized) or `/vehicle/velocity`, there will be zero compiler errors and zero runtime errors. The DDS network simply routes them as two completely independent paths. *Always use `ros2 topic list` and `ros2 topic echo <topic_name>` to verify data flow before doubting your code.*
- **Best Practice – Zero-Copy Pointers in C++:** Notice in the C++ subscriber, the callback argument is `const std_msgs::msg::Float32::SharedPtr msg`. We pass by a constant shared pointer. For a simple float, it doesn't matter. But if that message is a 15-megabyte PointCloud from a 3D LiDAR, passing by value would cause a massive memory copy, creating a catastrophic CPU spike. Always use pointers for message callbacks.
- **Pitfall – The Missing Placeholder in C++:** When binding the callback in `create_subscription`, engineers often forget the `std::placeholders::_1`. This tells C++ that the callback function expects exactly one argument (the incoming message). Without `_1`, the code will refuse to compile with a deeply confusing template error.

2.1.4 Part 4: Inspecting the Data Stream (CLI Tools)

The "Why"

Nodes are black boxes once compiled. If the vehicle's steering is erratic, you don't know if the perception node is calculating the wrong angles, or if the motor controller is just failing to execute them. You need to "tap the wire" and read the data mid-flight.

The "How" (Architecture Level)

Just like `ros2 node` commands, `ros2 topic` commands temporarily spin up an anonymous, hidden node in your terminal. This node subscribes to the requested topic, processes the incoming DDS packets (either printing them, timing them, or analyzing the metadata), and then cleanly shuts down when you press `Ctrl+C`.

1. The Typographical Check: `ros2 topic list -t`

The "Why": You need to know what streams are active on the network and, critically, what language (message type) they speak.

The "How": The `-t` flag (type) is mandatory for serious debugging. It forces the DDS discovery protocol to return not just the topic names, but the exact message interface associated with them.

Real-world Autonomous Racing Use Case: If your autonomous technical team is trying to integrate a new IMU driver, you run this to confirm whether the driver outputs standard `sensor_msgs/msg/Imu` data, or if the manufacturer used a proprietary custom

message type that will break your Extended Kalman Filter.

Implementation

Terminal

```
ros2 topic list -t

# Expected Terminal Output:
# /parameter_events [rcl_interfaces/msg/ParameterEvent]
# /rosout [rcl_interfaces/msg/Log]
# /vehicle/speed [std_msgs/msg/Float32]
```

2. The Wiretap: `ros2 topic echo <topic_name>`
The "Why": You need to see the actual payload. Are we publishing zeros? Are we publishing NaN because a matrix inversion failed in our C++ code?
The "How": This command forces your terminal to subscribe to the topic and print the deserialized data to standard output.
Real-world Autonomous Racing Use Case: The vehicle is refusing to accelerate. You echo the `/cmd_vel` (Command Velocity) topic. If the terminal prints `linear: {x: 0.0, y: 0.0, z: 0.0}`, you instantly know the problem is upstream in the planner, not a physical hardware failure in the motor.

Implementation

Terminal

```
ros2 topic echo /vehicle/speed

# Expected Terminal Output:
# data: 0.5
# ---
# data: 1.0
# ---
# data: 1.5
```

3. The Heartbeat Monitor: `ros2 topic hz <topic_name>`
The "Why": This is arguably the most important command for sensor integration. Algorithms are designed to expect data at specific frequencies. If the data is too slow, the system lags.
The "How": This command subscribes to the topic, records the timestamp of each incoming message, and calculates the moving average frequency (Hertz), min/max delay, and standard deviation.
Real-world Autonomous Racing Use Case: You configure a 3D LiDAR to spin at 20Hz. However, your team notices the obstacle avoidance is reacting sluggishly. You run `ros2 topic hz /scan`. The terminal shows an average rate of 4.2 Hz. You immediately know your network bandwidth is choked or the LiDAR driver is fundamentally misconfigured, dropping 75% of the frames.

Implementation

Terminal

```
ros2 topic hz /vehicle/speed

# Expected Terminal Output:
# average rate: 9.998
#   min: 0.100s max: 0.100s std dev: 0.00014s window: 10
```

4. The Graph Inspector: `ros2 topic info <topic_name> --verbose`
The "Why": You have verified a topic exists, but the subscriber node is still not receiving data. You need to see the "routing table" for that specific topic.
The "How": This command queries the ROS 2 graph to list exactly how many publishers and subscribers are attached to a topic. By appending `--verbose`, you force it to reveal the DDS Quality of Service (QoS) profiles for every connected node.
Real-world Autonomous Racing Use Case: You are trying to stream a high-resolution camera to a dashboard UI. The camera node is running, the UI node is running, but the screen is black. You run `ros2 topic info /camera/image_raw --verbose` and discover the Camera Publisher is set to Best Effort QoS, but the UI Subscriber is strictly demanding Reliable QoS. The middleware is silently blocking the connection.

Implementation

Terminal

```
ros2 topic info /vehicle/speed --verbose

# Expected Terminal Output:
# Type: std_msgs/msg/Float32
# Publisher count: 1
# Node name: speed_sensor_node
# Node namespace: /
# Topic type: std_msgs/msg/Float32
# Endpoint type: PUBLISHER
# GID: 01.0f.12...
# QoS profile:
#   Reliability: RELIABLE
#   History (Depth): KEEP_LAST (10)
#   Durability: VOLATILE
# Subscriber count: 1
# ...
```

> **Recommended Video Tutorial:**
>
> **How to Write C++ Subscriber and Publisher Nodes in ROS2 Humble**

from Scratch
This tutorial provides an excellent visual walkthrough of compiling and testing these exact C++ subscriber and publisher concepts in a live terminal environment.

2.2 Module 2.2: Services (Synchronous Requests)

2.2.1 Part 1: The "Why" & The "How" (Client/Server Architecture)

The "Why"
Topics use a "fire-and-forget" broadcast model. If you publish a message to enable the emergency brakes on a Topic, the Publisher has no idea if the motor controller actually received it or acted on it.

For critical, one-off commands, you need a Request/Response architecture. You need a guarantee. A Service allows one node (the **Client**) to send a specific Request to another node (the **Server**) and pause its own high-level logic until it receives a concrete Response (Success/Failure).

The "How" (Architecture Level)
Under the hood in the DDS layer, a ROS 2 Service is actually constructed using two hidden, paired DDS Topics.

- The Client publishes to a hidden `Request` topic and attaches a unique Sequence ID to the message.
- The Server processes the request and publishes to a hidden `Reply` topic, attaching that exact same Sequence ID.
- The Client matches the ID, accepts the response, and resumes its logic.

Unlike standard Topics, Services are 1-to-1 (or Many-to-1). Only one node can act as the Service Server, though multiple clients can send it requests.

Real-world Autonomous Racing Use Cases

- **Resetting the Extended Kalman Filter (EKF):** If your car spins out and the `robot_localization` node loses its pose estimation, you do not stream "reset" at 10Hz. You call an `/ekf/reset` service once, and the EKF replies "Reset Complete" so the planner knows it can safely resume navigation.
- **Sensor Calibration Routine:** Triggering the steering servo to sweep from lock-to-lock to find its center before the race begins.
- **State Management:** Changing the state of the PID controller (e.g., toggling it from Active to Inactive) using a boolean flag.

2.2.2 Part 2: Implementation - The Service Server

We will build a `PID_State_Manager` node. It will host a service that allows other nodes to turn the autonomous steering PID controller on or off. We will use the standard `std_srvs/srv/SetBool` interface.

Note: A ***`SetBool`*** *request contains a boolean* ***`data`****. The response contains a boolean* ***`success`*** *and a string* ***`message`****.*

Modern C++ 14/17 Implementation

File: ***`src/pid_server.cpp`***

```cpp
#include <memory>
#include "rclcpp/rclcpp.hpp"
// The specific service interface type
#include "std_srvs/srv/set_bool.hpp"

using std::placeholders::_1;
using std::placeholders::_2;

class PIDServer : public rclcpp::Node
{
public:
    PIDServer() : Node("pid_manager_node"), pid_active_(false)
    {
        // 1. Create the Service Server.
        // Syntax: create_service<ServiceType>("service_name", callback_function)
        service_ = this->create_service<std_srvs::srv::SetBool>(
            "/control/toggle_pid",
            std::bind(&PIDServer::handle_toggle_request, this, _1, _2));

        RCLCPP_INFO(this->get_logger(), "PID Manager Service Ready on /control/toggle_pid");
    }

private:
    // 2. The Service Callback.
    // It takes a Shared Pointer to the Request, and a Shared Pointer to the Response.
    void handle_toggle_request(
        const std::shared_ptr<std_srvs::srv::SetBool::Request> request,
        std::shared_ptr<std_srvs::srv::SetBool::Response> response)
    {
        // Act on the request
        pid_active_ = request->data;

        if (pid_active_) {
```

```
            RCLCPP_INFO(this->get_logger(), "Engaging PID
                Controller...");
            response->message = "PID Successfully Engaged.";
        } else {
            RCLCPP_WARN(this->get_logger(), "Disengaging PID
                Controller!");
            response->message = "PID Successfully Disengaged.";
        }

        // Populate the response
        response->success = true;

        // The callback finishes, and the Executor automatically
            sends the 'response' back to the Client.
    }

    rclcpp::Service<std_srvs::srv::SetBool>::SharedPtr service_;
    bool pid_active_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<PIDServer>());
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

File: ***pid_server.py***

```
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool

class PIDServer(Node):
    def __init__(self):
        super().__init__('pid_manager_node')
        self.pid_active_ = False

        # 1. Create the Service Server
        self.srv = self.create_service(
            SetBool,
            '/control/toggle_pid',
            self.handle_toggle_request)

        self.get_logger().info('PID Manager Service Ready on /
            control/toggle_pid')

    # 2. The Service Callback
    def handle_toggle_request(self, request, response):
        self.pid_active_ = request.data
```

```
21
22          if self.pid_active_:
23              self.get_logger().info('Engaging PID Controller...')
24              response.message = 'PID Successfully Engaged.'
25          else:
26              self.get_logger().warning('Disengaging PID Controller
                    !')
27              response.message = 'PID Successfully Disengaged.'
28
29          response.success = True
30
31          # In Python, you MUST return the response object
32          return response
33
34  def main(args=None):
35      rclpy.init(args=args)
36      node = PIDServer()
37      rclpy.spin(node)
38      node.destroy_node()
39      rclpy.shutdown()
40
41  if __name__ == '__main__':
42      main()
```

2.2.3 Part 3: Implementation - The Service Client (With Async Best Practices)

Now, we build a `Safety_Dashboard_Client`. It will request the PID to turn off.

Modern C++ 14/17 Implementation

File: src/safety_client.cpp

```
1  #include <chrono>
2  #include <memory>
3  #include "rclcpp/rclcpp.hpp"
4  #include "std_srvs/srv/set_bool.hpp"
5
6  using namespace std::chrono_literals;
7
8  class SafetyClient : public rclcpp::Node
9  {
10 public:
11     SafetyClient() : Node("safety_dashboard_node")
12     {
13         // 1. Create the Client
14         client_ = this->create_client<std_srvs::srv::SetBool>("/
               control/toggle_pid");
15
16         // 2. Wait for the server to actually exist on the
               network before sending requests
```

```
17          while (!client_->wait_for_service(1s)) {
18              if (!rclcpp::ok()) {
19                  RCLCPP_ERROR(this->get_logger(), "Interrupted
                        while waiting for the service. Exiting.");
20                  return;
21              }
22              RCLCPP_INFO(this->get_logger(), "Waiting for PID
                    Service to appear on network...");
23          }
24
25          send_disable_command();
26      }
27
28  private:
29      void send_disable_command()
30      {
31          // 3. Formulate the request
32          auto request = std::make_shared<std_srvs::srv::SetBool::
                Request>();
33          request->data = false; // Turn PID OFF
34
35          RCLCPP_WARN(this->get_logger(), "Sending EMERGENCY
                DISABLE command to PID...");
36
37          // 4. CRITICAL: Send the request Asynchronously.
38          // We bind a callback function that will execute ONLY
                when the response arrives.
39          client_->async_send_request(
40              request,
41              std::bind(&SafetyClient::response_callback, this, std
                    ::placeholders::_1));
42      }
43
44      // 5. The Response Callback
45      void response_callback(rclcpp::Client<std_srvs::srv::SetBool
            >::SharedFuture future)
46      {
47          // Extract the response from the future object
48          auto response = future.get();
49          if (response->success) {
50              RCLCPP_INFO(this->get_logger(), "Server replied: %s",
                        response->message.c_str());
51          } else {
52              RCLCPP_ERROR(this->get_logger(), "Failed to disable
                    PID!");
53          }
54      }
55
56      rclcpp::Client<std_srvs::srv::SetBool>::SharedPtr client_;
57  };
58
```

```cpp
int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<SafetyClient>());
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

File: safety_client.py

```python
import rclpy
from rclpy.node import Node
from std_srvs.srv import SetBool

class SafetyClient(Node):
    def __init__(self):
        super().__init__('safety_dashboard_node')

        # 1. Create the Client
        self.client = self.create_client(SetBool, '/control/
            toggle_pid')

        # 2. Wait for the server to exist
        while not self.client.wait_for_service(timeout_sec=1.0):
            self.get_logger().info('Waiting for PID Service to
                appear on network...')

        self.send_disable_command()

    def send_disable_command(self):
        # 3. Formulate the request
        request = SetBool.Request()
        request.data = False # Turn PID OFF

        self.get_logger().warning('Sending EMERGENCY DISABLE
            command to PID...')

        # 4. CRITICAL: Send the request Asynchronously.
        future = self.client.call_async(request)
        # Attach a callback to the future object
        future.add_done_callback(self.response_callback)

    # 5. The Response Callback
    def response_callback(self, future):
        try:
            response = future.result()
            if response.success:
                self.get_logger().info(f'Server replied: {
                    response.message}')
            else:
                self.get_logger().error('Failed to disable PID!')
```

```
        except Exception as e:
            self.get_logger().error(f'Service call failed: {e}')

def main(args=None):
    rclpy.init(args=args)
    node = SafetyClient()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2.2.4 Part 4: Best Practices & Pitfalls (CRITICAL: The Deadlock Trap)

This is the most common reason senior engineers have to rescue junior engineers' code.

The Deadly Pitfall: Blocking the Executor (Deadlocks)
In ROS 1, it was common to make a synchronous service call (`client->send_request()` in C++ or `client.call()` in Python) directly inside a timer callback. In ROS 2, if you do this while using the default Single-Threaded Executor, your entire node will permanently freeze in a deadlock.
Why does this happen?

1. Your timer fires. The single Executor thread enters the timer callback.
2. Inside the callback, you send a synchronous request to the Server and wait. You are telling the thread, "Do not move until the response arrives."
3. The Server processes the request and sends the response back over the DDS network.
4. The response arrives at your Client node, waiting in the queue.

The Trap: The only thread capable of reading that response queue is currently frozen inside your timer callback, waiting for the response! The system is permanently deadlocked.

How Professionals Handle This:

- **Always use Asynchronous calls (`async_send_request` / `call_async`):** As demonstrated in the code above, you fire the request and immediately exit the function. You provide a callback that the Executor will run only when the response naturally arrives in the queue.
- **Callback Groups & Multi-Threaded Executors:** If you are writing a complex state machine (like an autonomous mission planner) where you absolutely *must* block and wait sequentially, you must place the Service Client into a separate `MutuallyExclusiveCallbackGroup` and spin your node using an `rclcpp::executors::MultiThreadedExecutor`. (We will cover this deeply in Phase 5: Advanced Node Architecture).

2.3 Module 2.3: Actions (Asynchronous, Long-Running Tasks)

2.3.1 Part 1: The "Why" & The "How" (Proactive Cancellation)

The "Why"
Your high-level planner (the Client) is the brain. The Action Server is the muscle. If the brain's perception nodes detect a sudden obstacle at 25 meters, the brain must instantly instruct the muscle to abort the 50-meter drive command.

The "How" (Architecture Level)
When an Action Client sends a goal, it receives a Goal Handle. This handle is your direct line of communication to that specific executing task. To cancel, the Client takes that Goal Handle and sends an asynchronous request to the Server's hidden Cancel Service. The Server processes it, safely winds down its hardware loop (as we coded in the previous module), and returns a `CANCELED` result to the Client.

Real-world Autonomous Racing Use Cases

- **Dynamic Obstacle Avoidance:** Aborting a path-following action because a slower car swerved into your lane.
- **Emergency Pit Stop:** Canceling an active track lap action to immediately route the vehicle back to the pit lane due to low battery voltage.

2.3.2 Part 2: Implementation - The Action Client (With Safety Preemption)

We will build the `SafetyPlannerClient`. It will request a 50-meter drive. When the feedback indicates we have reached 25 meters, it will simulate detecting an obstacle and trigger the cancellation.

Modern C++ 14/17 Implementation

File: src/safety_planner_client.cpp

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
#include "racing_interfaces/action/drive_distance.hpp"

using namespace std::placeholders;

class SafetyPlannerClient : public rclcpp::Node
{
public:
    using DriveDistance = racing_interfaces::action::
        DriveDistance;
```

```
    using GoalHandleDrive = rclcpp_action::ClientGoalHandle<
        DriveDistance>;

    SafetyPlannerClient() : Node("safety_planner_client")
    {
        // 1. Create the Action Client
        this->client_ptr_ = rclcpp_action::create_client<
            DriveDistance>(
            this, "/control/drive_distance");

        // Wait for server to appear on the network
        while (!this->client_ptr_->wait_for_action_server(std::
            chrono::seconds(1))) {
            RCLCPP_INFO(this->get_logger(), "Waiting for Drive
                Action Server...");
        }

        // Send the command immediately for this example
        this->send_goal();
    }

private:
    rclcpp_action::Client<DriveDistance>::SharedPtr client_ptr_;
    bool cancel_triggered_ = false; // Guard flag to ensure we
        only send cancel once

    void send_goal()
    {
        RCLCPP_INFO(this->get_logger(), "Planner requesting 50-
            meter drive.");

        auto goal_msg = DriveDistance::Goal();
        goal_msg.target_distance = 50.0;

        auto send_goal_options = rclcpp_action::Client<
            DriveDistance>::SendGoalOptions();

        // Bind our 3 crucial async callbacks
        send_goal_options.goal_response_callback =
            std::bind(&SafetyPlannerClient::
                goal_response_callback, this, _1);
        send_goal_options.feedback_callback =
            std::bind(&SafetyPlannerClient::feedback_callback,
                this, _1, _2);
        send_goal_options.result_callback =
            std::bind(&SafetyPlannerClient::result_callback, this
                , _1);

        // 2. Send the Goal Asynchronously
        this->client_ptr_->async_send_goal(goal_msg,
            send_goal_options);
```

```
52      }
53
54      // Callback 1: Did the server accept our request?
55      void goal_response_callback(const GoalHandleDrive::SharedPtr
            & goal_handle)
56      {
57          if (!goal_handle) {
58              RCLCPP_ERROR(this->get_logger(), "Goal was rejected
                    by server");
59          } else {
60              RCLCPP_INFO(this->get_logger(), "Goal accepted by
                    server, waiting for result");
61          }
62      }
63
64      // Callback 2: Continuous Feedback Stream & The Safety
            Trigger
65      void feedback_callback(
66          GoalHandleDrive::SharedPtr goal_handle,
67          const std::shared_ptr<const DriveDistance::Feedback>
                feedback)
68      {
69          RCLCPP_INFO(this->get_logger(), "Feedback: Driven %.2f
                meters", feedback->distance_traveled);
70
71          // --- THE PREEMPTION TRIGGER ---
72          if (feedback->distance_traveled >= 25.0 && !this->
                cancel_triggered_) {
73              RCLCPP_WARN(this->get_logger(), "OBSTACLE DETECTED!
                    Sending Cancel Request...");
74
75              // Asynchronously send the cancel request
76              this->client_ptr_->async_cancel_goal(goal_handle);
77              this->cancel_triggered_ = true; // Prevent spamming
                    cancel requests
78          }
79      }
80
81      // Callback 3: The Final Outcome
82      void result_callback(const GoalHandleDrive::WrappedResult &
            result)
83      {
84          switch (result.code) {
85              case rclcpp_action::ResultCode::SUCCEEDED:
86                  RCLCPP_INFO(this->get_logger(), "Goal succeeded!
                        Time: %.2f", result.result->total_time_elapsed
                        );
87                  break;
88              case rclcpp_action::ResultCode::ABORTED:
89                  RCLCPP_ERROR(this->get_logger(), "Goal was
                        aborted by server.");
```

```
                    break;
                case rclcpp_action::ResultCode::CANCELED:
                    // We expect this branch to trigger due to our
                       preemption!
                    RCLCPP_WARN(this->get_logger(), "Goal
                       successfully canceled by planner intervention!
                       ");
                    break;
                default:
                    RCLCPP_ERROR(this->get_logger(), "Unknown result
                       code");
                    break;
            }
            rclcpp::shutdown();
        }
};

int main(int argc, char ** argv){
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<SafetyPlannerClient>());
    return 0;
}
```

Python Implementation

File: safety_planner_client.py

```
import rclpy
from rclpy.node import Node
from rclpy.action import ActionClient
from racing_interfaces.action import DriveDistance

class SafetyPlannerClient(Node):
    def __init__(self):
        super().__init__('safety_planner_client')

        # 1. Create the Action Client
        self._action_client = ActionClient(self, DriveDistance, '
           /control/drive_distance')
        self.current_goal_handle = None
        self.cancel_triggered_ = False

        self.send_goal()

    def send_goal(self):
        self.get_logger().info('Waiting for Drive Action Server
           ...')
        self._action_client.wait_for_server()

        goal_msg = DriveDistance.Goal()
        goal_msg.target_distance = 50.0

```

```
24          self.get_logger().info('Planner requesting 50-meter drive
               .')
25
26          # 2. Send the Goal Asynchronously
27          self._send_goal_future = self._action_client.
               send_goal_async(
28              goal_msg, feedback_callback=self.feedback_callback)
29
30          # Attach callback for when the server accepts/rejects the
                goal
31          self._send_goal_future.add_done_callback(self.
               goal_response_callback)
32
33      # Callback 1: Goal Acceptance
34      def goal_response_callback(self, future):
35          goal_handle = future.result()
36          if not goal_handle.accepted:
37              self.get_logger().error('Goal rejected by server.')
38              return
39
40          self.get_logger().info('Goal accepted by server, waiting
               for result.')
41          self.current_goal_handle = goal_handle # Store handle for
                cancellation
42
43          # Attach callback for the final result
44          self._get_result_future = goal_handle.get_result_async()
45          self._get_result_future.add_done_callback(self.
               get_result_callback)
46
47      # Callback 2: Continuous Feedback Stream & The Safety Trigger
48      def feedback_callback(self, feedback_msg):
49          distance = feedback_msg.feedback.distance_traveled
50          self.get_logger().info(f'Feedback: Driven {distance:.2f}
               meters')
51
52          # --- THE PREEMPTION TRIGGER ---
53          if distance >= 25.0 and not self.cancel_triggered_ and
               self.current_goal_handle:
54              self.get_logger().warning('OBSTACLE DETECTED! Sending
                    Cancel Request...')
55
56              # Asynchronously send the cancel request
57              self.current_goal_handle.cancel_goal_async()
58              self.cancel_triggered_ = True # Prevent spamming
59
60      # Callback 3: The Final Outcome
61      def get_result_callback(self, future):
62          result_status = future.result().status
63          from action_msgs.msg import GoalStatus
64
```

```
65          if result_status == GoalStatus.STATUS_SUCCEEDED:
66              self.get_logger().info('Goal succeeded!')
67          elif result_status == GoalStatus.STATUS_CANCELED:
68              # We expect this branch to trigger due to our
                    preemption!
69              self.get_logger().warning('Goal successfully canceled
                    by planner intervention!')
70          elif result_status == GoalStatus.STATUS_ABORTED:
71              self.get_logger().error('Goal aborted by server.')
72
73          rclpy.shutdown()
74
75  def main(args=None):
76      rclpy.init(args=args)
77      node = SafetyPlannerClient()
78      rclpy.spin(node)
79
80  if __name__ == '__main__':
81      main()
```

2.3.3 Part 3: Best Practices & Pitfalls

Critical Action Client Pitfalls

- **The "Spamming Cancel" Pitfall:** Notice the `cancel_triggered_` boolean flag in both implementations. A common mistake is putting the cancel logic inside the feedback callback without a guard flag. Because feedback streams at high frequencies, the moment the vehicle crosses 25 meters, the Client will rapidly fire hundreds of `async_cancel_goal` requests over the DDS network before the Server has time to respond and shut down. This can completely choke the middleware. Always use a state flag.
- **Losing the Goal Handle (Python):** In C++, the `feedback_callback` conveniently provides the `goal_handle` as an argument. In Python, the `feedback_callback` only receives the message. You must store `self.current_goal_handle` during the `goal_response_callback` so you have a reference to it when you need to cancel later.
- **Handling Server Rejection:** Never assume the server will accept your goal. If the car is currently in an "Emergency Stop" state, the Server might reply with `REJECT` to your drive command. If your Client code immediately tries to ask for a result on a rejected goal, it will throw a fatal null pointer exception. Always check if the goal was accepted before proceeding.

2.3.4 Part 4: Inspecting the Action Graph (CLI Tools)

The "Why"

When your vehicle refuses to move, or it starts moving and won't stop, you need to know immediately: Is the Server dead? Is the Client sending garbage data? Did the Server reject the goal? The Action CLI tools act as a temporary diagnostic Client/Server to give you instant answers on the track.

The "How" (Architecture Level)
Because an Action is secretly composed of five different Topics and Services, looking at raw `ros2 topic list` output when an Action is running is a nightmare—you will just see a massive wall of hidden `/control/drive_distance/_action/status`, `/feedback`, and `/send_goal` topics.

When you use `ros2 action` commands, the CLI wraps these complex underlying DDS calls into a clean, human-readable terminal output. It handles the asynchronous futures and state tracking for you behind the scenes.

1. The Interface Verification: `ros2 action list -t`
The "Why": Before you can send a goal or debug a connection, you need to know exactly what Actions are exposed to the network and what `.action` interface they speak.
The "How": Just like with topics, the `-t` flag forces the middleware to return the interface type.
Real-world Autonomous Racing Use Case: You just launched the massive Nav2 (Navigation2) stack. Before you try to write a high-level racing logic script, you run this command to verify that Nav2 successfully brought up the `/navigate_to_pose` Action Server and is ready to accept waypoints.

Implementation

Terminal

```
ros2 action list -t

# Expected Terminal Output:
# /control/drive_distance [racing_interfaces/action/DriveDistance]
# /navigate_to_pose [nav2_msgs/action/NavigateToPose]
```

2. The Graph & State Inspector: `ros2 action info <action_name>`
The "Why": You sent a goal to drive 50 meters, but the car is just sitting there. You need to know if the Action Server is actually online and if it's currently processing any goals.
The "How": This command queries the ROS 2 graph specifically for Action clients and servers attached to that namespace, and tells you how many nodes are connected.
Real-world Autonomous Racing Use Case: If you run this command and see `Action Servers: 0`, you instantly know your `drive_action_server` crashed or was never launched. If you see `Action Servers: 1`, but the car isn't moving, you know the problem is inside your goal logic, not the network.

Implementation

Terminal

```
ros2 action info /control/drive_distance

# Expected Terminal Output:
# Action: /control/drive_distance
# Action clients: 1
#     /safety_planner_client
# Action servers: 1
#     /drive_action_server
```

3. The Manual Trigger: `ros2 action send_goal` (with Feedback)
The "Why": This is your most powerful hardware testing tool. Before you write a complex 200-line C++ Action Client, you want to manually test the Action Server. Does the motor actually spin when it receives a 5-meter drive command?
The "How": This command turns your terminal into a temporary Action Client. You pass the Action Name, the Interface Type, and the Goal Data formatted as a YAML string. Crucially, adding the `-f` (or `--feedback`) flag tells the terminal to dynamically subscribe to the hidden Feedback topic and print the stream to your screen while the task executes.
Real-world Autonomous Racing Use Case: Your car is jacked up in the pit garage. You want to test the low-level PID motor controllers without running the entire autonomous AI stack. You send a manual goal of 5 meters from your laptop and watch the feedback stream to ensure the wheel encoders are accurately reporting the distance traveled.

Implementation

Terminal

```
# Syntax: ros2 action send_goal <action_name> <action_type>
"{yaml_goal_data}" -f

ros2 action send_goal /control/drive_distance
racing_interfaces/action/DriveDistance "{target_distance: 5.0}"
-f

# Expected Terminal Output:
# Waiting for an action server to become available...
# Sending goal:
#      target_distance: 5.0
#
# Goal accepted with ID: 1e4b3c...
#
# Feedback:
#     distance_traveled: 1.0
# Feedback:
#     distance_traveled: 2.0
# Feedback:
#     distance_traveled: 3.0
# Feedback:
#     distance_traveled: 4.0
#
# Result:
#     total_time_elapsed: 5.0
#
# Goal finished with status: SUCCEEDED
```

Tech Lead Note:
If you don't use the `-f` flag, the terminal will just sit completely blank after "Goal accepted" until the final Result arrives. Always use `-f` during testing so you know the system hasn't frozen.

CHAPTER 3

Phase 3: The Command Center (Orchestration Tools)

3.1 Module 3.1: Parameters (Dynamic Configuration)

3.1.1 Part 1: The "Why" & The "How" (Distributed Parameters)

The "Why"
Parameters are configuration values associated with a node. They allow you to write generic, reusable code. Instead of writing a "Front Left Motor Node" and a "Rear Right Motor Node," you write one "Motor Node" and pass it a parameter indicating its physical location and max speed limits at startup.

The "How" (Architecture Level - A Massive Shift from ROS 1)
In ROS 1, there was a global "Parameter Server" hosted by the `roscore` master. If a node needed a parameter, it asked the master. This was a massive bottleneck and a single point of failure.

In ROS 2, **there is no central Parameter Server**. Instead, *every single node is its own parameter server*. Under the hood, when you instantiate an `rclcpp::Node` or `rclpy.node.Node`, the ROS 2 middleware automatically attaches several hidden Services to it (e.g., `~/get_parameters`, `~/set_parameters`). When you change a parameter from the terminal, you are actually just making a synchronous Service call to that specific node.

Real-world Autonomous Racing Use Cases

- **PID Tuning:** Adjusting Proportional, Integral, and Derivative gains while the car is actively driving.
- **Hardware Configuration:** Setting the camera resolution (e.g., 720p vs 1080p) or LiDAR rotation frequency based on the specific track environment.
- **Debug Toggles:** Toggling a boolean `publish_debug_images` parameter. When

true, the node publishes heavy, annotated images for RViz. When false, it saves CPU cycles during the actual race.

3.1.2 Part 2: Implementation - Dynamic Parameters (The Code)

We will build a `PidControllerNode`. It will declare three parameters (`kp`, `ki`, `kd`). **Crucially**, we will implement a Parameter Event Callback so that if the pit crew changes the gains from the terminal, the node instantly updates its internal math without restarting.

Modern C++ 14/17 Implementation

File: src/pid_controller_node.cpp

```cpp
#include <memory>
#include <vector>
#include "rclcpp/rclcpp.hpp"
#include "rcl_interfaces/msg/set_parameters_result.hpp"

class PidControllerNode : public rclcpp::Node
{
public:
    PidControllerNode() : Node("pid_controller")
    {
        // 1. Declare the parameters and their default values.
        // In ROS 2 Humble, you MUST declare a parameter before you can use or set it.
        this->declare_parameter("kp", 1.0);
        this->declare_parameter("ki", 0.0);
        this->declare_parameter("kd", 0.1);

        // 2. Fetch the initial values into our class variables
        kp_ = this->get_parameter("kp").as_double();
        ki_ = this->get_parameter("ki").as_double();
        kd_ = this->get_parameter("kd").as_double();

        RCLCPP_INFO(this->get_logger(), "PID Initialized with Kp: %.2f, Ki: %.2f, Kd: %.2f", kp_, ki_, kd_);

        // 3. THE CRITICAL STEP: Register the Dynamic Parameter Callback
        // This tells the Executor: "If anyone modifies a parameter via CLI, run this function."
        param_subscriber_ = this->add_on_set_parameters_callback(
            std::bind(&PidControllerNode::param_callback, this, std::placeholders::_1));
    }

private:
    double kp_, ki_, kd_;
    // We must keep a shared pointer to the callback handle, or it will be destroyed out of scope!
```

```
    rclcpp::node_interfaces::OnSetParametersCallbackHandle::
        SharedPtr param_subscriber_;

    // 4. The Parameter Callback Function
    rcl_interfaces::msg::SetParametersResult param_callback(const
        std::vector<rclcpp::Parameter> & params)
    {
        rcl_interfaces::msg::SetParametersResult result;
        result.successful = true;
        result.reason = "Success";

        // Iterate through the list of parameters that were just
            changed
        for (const auto & param : params) {
            if (param.get_name() == "kp") {
                // Safety Check: We can reject invalid values!
                if (param.as_double() < 0.0) {
                    result.successful = false;
                    result.reason = "Kp cannot be negative!";
                    return result;
                }
                kp_ = param.as_double();
                RCLCPP_INFO(this->get_logger(), "Dynamically
                    updated Kp to: %.2f", kp_);
            }
            else if (param.get_name() == "ki") {
                ki_ = param.as_double();
                RCLCPP_INFO(this->get_logger(), "Dynamically
                    updated Ki to: %.2f", ki_);
            }
            else if (param.get_name() == "kd") {
                kd_ = param.as_double();
                RCLCPP_INFO(this->get_logger(), "Dynamically
                    updated Kd to: %.2f", kd_);
            }
        }
        return result; // Tell the parameter server (and the CLI)
            that the change was accepted
    }
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<PidControllerNode>());
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

File: ***pid_controller_node.py***

```python
import rclpy
from rclpy.node import Node
from rcl_interfaces.msg import SetParametersResult

class PidControllerNode(Node):
    def __init__(self):
        super().__init__('pid_controller')

        # 1. Declare parameters and defaults
        self.declare_parameter('kp', 1.0)
        self.declare_parameter('ki', 0.0)
        self.declare_parameter('kd', 0.1)

        # 2. Fetch initial values
        self.kp_ = self.get_parameter('kp').value
        self.ki_ = self.get_parameter('ki').value
        self.kd_ = self.get_parameter('kd').value

        self.get_logger().info(f'PID Initialized with Kp: {self.kp_}, Ki: {self.ki_}, Kd: {self.kd_}')

        # 3. Register the Dynamic Parameter Callback
        self.add_on_set_parameters_callback(self.param_callback)

    # 4. The Parameter Callback Function
    def param_callback(self, params):
        successful = True
        reason = "Success"

        for param in params:
            if param.name == 'kp':
                if param.value < 0.0:
                    return SetParametersResult(successful=False, reason="Kp cannot be negative!")
                self.kp_ = param.value
                self.get_logger().info(f'Dynamically updated Kp to: {self.kp_}')
            elif param.name == 'ki':
                self.ki_ = param.value
                self.get_logger().info(f'Dynamically updated Ki to: {self.ki_}')
            elif param.name == 'kd':
                self.kd_ = param.value
                self.get_logger().info(f'Dynamically updated Kd to: {self.kd_}')

        return SetParametersResult(successful=successful, reason=reason)
```

```
43
44 def main(args=None):
45     rclpy.init(args=args)
46     node = PidControllerNode()
47     rclpy.spin(node)
48     rclpy.shutdown()
49
50 if __name__ == '__main__':
51     main()
```

3.1.3 Part 3: Bulk Loading (YAML Configurations)

When your autonomous stack grows, you will have hundreds of parameters (max velocity, acceleration limits, LiDAR topics, TF frame names). You cannot set these one-by-one in the terminal. You use a YAML file.

1. Create a file `pid_config.yaml`:
Notice how we namespace the parameters under the exact name of the node (`pid_controller`):

```
1 pid_controller:
2   ros__parameters:
3     kp: 2.5
4     ki: 0.1
5     kd: 0.05
6     max_steering_angle: 0.35
```

2. Inject the YAML at Startup:
You use the `--ros-args --params-file` flags to force the node to read the YAML file instead of its hardcoded defaults when it boots up.

Terminal

```
ros2 run my_control_pkg pid_controller_node --ros-args --params-file
./pid_config.yaml
```

3.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Parameter Pitfalls

- **The "Parameter Not Declared" Exception:** In older versions of ROS 2 (like Foxy), you could get or set a parameter even if the node didn't know about it. In ROS 2 Humble, this is strictly forbidden. If you try to run `this->get_parameter("max_speed")` without calling `this->declare_parameter("max_speed")` first, the node will instantly crash with an exception. *Always declare your parameters.*
- **Rejecting Bad Data (The Safety Net):** As shown in the C++ callback, the `SetParametersResult` is not just a formality. If a junior engineer types `ros2 param set /pid_controller kp -5.0` in the terminal, your node must catch that negative value and return `successful = false`. The CLI will literally reject the engineer's command on the terminal screen, and the car remains safe.

- **Dangling Callback Pointers (C++ Memory Leak):** If you do not store the result of `add_on_set_parameters_callback` into a class member variable (like `param_subscriber_`), the shared pointer will be destroyed as soon as the constructor finishes. The callback will silently detach, and your dynamic updates will magically stop working.

3.1.5 Part 5: Inspecting and Mutating Parameters (CLI Tools)

The "Why"
During live track testing, the vehicle's behavior is a physical manifestation of its internal math. When the car understeers, the pit crew needs to surgically alter the steering gains without halting the vehicle's heartbeat.

The "How"
These CLI commands are actually making hidden Service calls over the DDS network directly to the `~/get_parameters` and `~/set_parameters` interfaces automatically attached to every ROS 2 node.

1. The Audit: `ros2 param list`
Before you change a variable, you need to know exactly what the node exposes.

Terminal
```
ros2 param list

# Expected Terminal Output:
# /pid_controller:
#   kp
#   ki
#   kd
#   use_sim_time
```

2. The Probe: `ros2 param get <node_name> <parameter_name>`
Read the exact value currently residing in the node's memory.

Terminal
```
ros2 param get /pid_controller kp

# Expected Terminal Output:
# Double value is: 1.0
```

3. The Mutation: `ros2 param set <node_name> <parameter_name> <value>`
This is the command that triggers the C++ or Python callback we just wrote.

Terminal

```
ros2 param set /pid_controller kp 1.5

# Expected Terminal Output:
# Set parameter successful

# (If you try to set a negative value based on our safety check):
ros2 param set /pid_controller kp -2.0

# Setting parameter failed: Kp cannot be negative!
```

4. The Save-State (CRITICAL): `ros2 param dump <node_name>`
This is the most important command for the track. After 45 minutes of grueling track testing, your vehicle is driving perfectly. You have manually tuned `kp`, `ki`, and `kd` via the terminal. But the moment you hit `Ctrl+C` to shut down the car, those perfectly tuned variables vanish from RAM. You must dump them to a file before shutdown so you can load them on the next boot.

Terminal

```
# Dump the live configuration directly into a YAML file
ros2 param dump /pid_controller > perfectly_tuned_pid.yaml

# You can then use this file on the next boot:
# ros2 run my_control_pkg pid_controller_node --ros-args
--params-file ./perfectly_tuned_pid.yaml
```

3.2 Module 3.2: The Launch System

The "Why"
A production autonomous stack consists of dozens, if not hundreds, of nodes: LiDAR drivers, camera drivers, IMU filters, localization algorithms, local planners, global planners, and motor controllers.

Opening 50 terminals and typing `ros2 run ... --ros-args --params-file ...` is impossible. We need a deterministic, repeatable, and programmable way to bring up the entire system with a single command.

The "How" (Architecture Level)
In ROS 2, the Launch system is written entirely in Python. This is a massive upgrade from the static XML files of ROS 1. Because launch files are Python scripts, they are Turing-complete. You can use if/else logic, read environment variables, dynamically discover file paths, and conditionally launch different nodes based on the hardware you are running (e.g., "If $SIMULATION is true, launch Gazebo; else, launch the physical hardware drivers").

Real-world Autonomous Racing Use Cases

- **The "Bringup" Script:** A single `racecar_bringup.launch.py` that starts the perception stack, loads all the tuned YAML files, and boots the motor controllers.
- **Topic Remapping:** You download a community LiDAR package that strictly publishes to `/laser_scan`, but your navigation stack expects `/scan`. You use the launch file to silently "remap" the topic name without touching the C++ source code.
- **Namespacing:** Running two identical race cars on the same Wi-Fi network. You use a launch file to push Car 1's nodes into the `/car1/` namespace and Car 2 into `/car2/` so their topics don't collide.

3.2.1 Implementation: The Python Launch File

Unlike Nodes, Launch files do not have a C++ equivalent. All launch files in ROS 2 are Python. They can launch both C++ and Python executables seamlessly.

We will create a file that launches our `TelemetryNode` (from Module 1), our `PidControllerNode` (from Module 3.1), loads a YAML file, and remaps a topic.

File: *`launch/racecar_bringup.launch.py`*

```
import os
from ament_index_python.packages import
    get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    # 1. Dynamically find the path to our tuned YAML file
```

```
    # This prevents hardcoding absolute paths like /home/user/
        ros2_ws/...
    control_pkg_dir = get_package_share_directory('my_control_pkg
        ')
    pid_config_path = os.path.join(control_pkg_dir, 'config', '
        perfectly_tuned_pid.yaml')

    # 2. Define Node 1: The Telemetry Monitor (A simple launch)
    telemetry_node = Node(
        package='my_cpp_pkg',           # The package name
        executable='telemetry_node',    # The executable name
        name='telemetry_monitor',       # The node name on the
            graph
        output='screen'                 # Print logs directly to
            the terminal
    )

    # 3. Define Node 2: The PID Controller (Advanced launch)
    pid_node = Node(
        package='my_control_pkg',
        executable='pid_controller_node',
        name='pid_controller',
        output='screen',

        # A. Load the parameters from the YAML file we dumped
            earlier
        parameters=[pid_config_path],

        # B. Remap a topic:
        # The node was written to publish to "/cmd_vel", but we
            want it on "/steering/cmd_vel"
        remappings=[
            ('/cmd_vel', '/steering/cmd_vel')
        ]
    )

    # 4. Return the LaunchDescription
    # The Launch Executor evaluates this object and boots the
        nodes.
    return LaunchDescription([
        telemetry_node,
        pid_node
    ])
```

Executing the Launch File

You run this script using the `ros2 launch` command.

Terminal

```
# Syntax: ros2 launch <package_name> <launch_file_name>

ros2 launch my_control_pkg racecar_bringup.launch.py
```

Best Practices & Common Pitfalls

- **Pitfall – Hardcoding File Paths:** Never use `parameters=['/home/ammar/ros2_ws/src/.../config.yaml']`. If another engineer pulls your code on their machine, it will instantly crash because their username isn't `ammar`. Always use `get_package_share_directory()` to resolve paths dynamically based on the active ROS 2 installation.
- **Pitfall – Forgetting to Install the Launch/Config Files:** If you put a `.launch.py` or `.yaml` file in your C++ package's `src` folder, `colcon build` will completely ignore it unless you explicitly tell CMake to install it.
  - **C++ fix:** Add `install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})` to your `CMakeLists.txt`.
  - **Python fix:** Add the paths to the `data_files` array in `setup.py`.
- **Best Practice – Launch File Composition:** Do not write one massive 500-line launch file. Write a `sensors.launch.py`, a `navigation.launch.py`, and a `control.launch.py`. Then, write a master `system.launch.py` that simply includes the others using `IncludeLaunchDescription`. This keeps the architecture modular.

3.2.2 Part 2: Launch File Composition (The Master Bringup)

The "Why"

We need a hierarchy. Sub-teams own their specific subsystem launch files (e.g., `sensors.launch.py`, `control.launch.py`). The Tech Lead owns the Master Bringup (`system.launch.py`), which acts as an orchestrator, simply pointing to the subsystem files and passing global configurations down the tree.

The "How" (Architecture Level)

Under the hood, the ROS 2 Launch framework evaluates the Python script to build a massive asynchronous execution tree. When you use `IncludeLaunchDescription`, the Executor dynamically finds the child Python script, parses its execution tree, and grafts it onto the master tree.

To pass data between these trees, we use Launch Arguments (`DeclareLaunchArgument` to create them, and `LaunchConfiguration` to read/pass them).

Real-world Autonomous Racing Use Cases

- **The Simulation Toggle (`use_sim_time`):** You test your autonomous software in a Gazebo simulation on Monday, and deploy it to the physical race car on Tuesday. You cannot afford to manually change 50 configuration files from "simulated time" to "hardware time." You toggle a single boolean at the top-level launch command, and it cascades down to every single node in the fleet.

Implementation: The Master `system.launch.py`

Assume the Perception team has written `sensors.launch.py` (in `my_sensor_pkg`) and the Control team has written `control.launch.py` (in `my_control_pkg`). Here is how you, as the Tech Lead, stitch them together.

File: ***`launch/system.launch.py`***

```python
import os
from ament_index_python.packages import
    get_package_share_directory

# Core Launch Modules
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription,
    DeclareLaunchArgument
from launch.launch_description_sources import
    PythonLaunchDescriptionSource
from launch.substitutions import LaunchConfiguration

def generate_launch_description():

    # 1. Define the Launch Configuration variables
    # This creates a variable that can hold the value passed from
        the terminal
    use_sim_time = LaunchConfiguration('use_sim_time')

    # 2. Declare the Launch Argument
    # This exposes the argument to the CLI so the engineer can
       type:
    # ros2 launch my_bringup_pkg system.launch.py use_sim_time:=
       true
    declare_use_sim_time_cmd = DeclareLaunchArgument(
        'use_sim_time',
        default_value='false',
        description='Use simulation (Gazebo) clock if true'
    )

    # 3. Locate the subsystem launch files dynamically
    sensor_pkg_dir = get_package_share_directory('my_sensor_pkg')
    control_pkg_dir = get_package_share_directory('my_control_pkg
       ')

    sensor_launch_path = os.path.join(sensor_pkg_dir, 'launch', '
       sensors.launch.py')
    control_launch_path = os.path.join(control_pkg_dir, 'launch',
        'control.launch.py')

    # 4. Include the Sensor Subsystem
    include_sensors = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(sensor_launch_path),
        # CRITICAL: Pass the global argument down to the child
           launch file
```

```
        launch_arguments={'use_sim_time': use_sim_time}.items()
    )

    # 5. Include the Control Subsystem
    include_control = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(control_launch_path),
        # Pass the global argument down here as well
        launch_arguments={'use_sim_time': use_sim_time}.items()
    )

    # 6. Build and return the Master Launch Tree
    return LaunchDescription([
        declare_use_sim_time_cmd,
        include_sensors,
        include_control
    ])
```

Executing the Master File with Arguments

Now, your engineers only need to memorize one command for the entire vehicle:

Terminal

```
# To run on the physical car:
ros2 launch my_bringup_pkg system.launch.py

# To run the exact same stack in Gazebo, synchronized to simulation
time:
ros2 launch my_bringup_pkg system.launch.py use_sim_time:=true
```

Best Practices & Common Pitfalls

- **The Silent Argument Drop (The Deadliest Pitfall):** If the master launch file passes `use_sim_time` down, but the child `sensors.launch.py` forgets to include its own `DeclareLaunchArgument('use_sim_time')`, the child file will silently ignore the variable and default to hardware time. Your LiDAR will stamp its data with the wrong timestamps, and the navigation stack will instantly crash due to TF2 extrapolation errors. Arguments must be explicitly declared in every file that expects to use them.
- **Best Practice – Namespacing Entire Subsystems:** If you ever need to run two vehicles simultaneously, you do not rewrite the code. You wrap the `IncludeLaunchDescription` inside a `PushRosNamespace` action. This will dynamically prefix every node and topic inside that child launch file with `/car1` or `/car2`, completely isolating their DDS traffic.

3.3 Module 3.3: Debugging and Data Logging

3.3.1 Part 1: The "Why" & The "How" (The Black Box & The Visualizer)

The "Why"
You cannot debug an autonomous system by reading raw text streams of float arrays. A `PointCloud2` message from a 3D LiDAR contains hundreds of thousands of XYZ coordinates per second. To understand why the perception algorithm failed, you need a "flight data recorder" (`rosbag2`) to perfectly capture the event, and a 3D GUI (`RViz2`) to visually reconstruct the robot's environment exactly as the algorithms saw it.

The "How" (Architecture Level)

- **`rosbag2`:** Under the hood, this is a highly optimized C++ node that dynamically subscribes to the topics you specify. Instead of processing the messages, it serializes the raw DDS payloads directly into a local database (SQLite3 is the default in ROS 2 Humble). Crucially, it logs the exact nanosecond timestamp of arrival. When you replay the bag, it publishes those messages back onto the network mimicking the exact original timing.
- **`RViz2` (ROS Visualization):** `RViz2` is not a simulator. It has no physics engine. It is strictly a passive listener. It subscribes to standard sensor topics and renders them in a 3D OpenGL window. It maps the data against the robot's kinematic coordinate frames (TF2) to show you where the sensor data physically exists relative to the car.

Real-world Autonomous Racing Use Cases

- **Debugging Phantom Braking:** The car emergency-brakes on an empty track. You replay the rosbag through `RViz2` and see that a single noisy point from the LiDAR bounced off a dust cloud, and the Nav2 local costmap incorrectly inflated it into a concrete wall.
- **Off-Track Tuning:** You want to tune the DWB (Dynamic Window Approach) critic weights in your local planner. Instead of draining the vehicle's battery by running it all day, you record one lap. You sit in the lab, loop the bag file, and watch the planner's projected trajectories instantly shift in `RViz2` as you mutate the parameters via the CLI.

3.3.2 Part 2: Implementation (CLI Tools)

1. The Flight Recorder: `rosbag2 record`
You use this command while the system is live on the track.

Recording Specific Topics (Recommended):
You explicitly tell the bag which streams to capture.

Terminal
```
# Syntax: rosbag2 record <topic1> <topic2> -o
<output_directory_name>

rosbag2 record /scan /odom /tf /tf_static /vehicle/speed -o
lap_3_telemetry
```

Recording Everything (Use with Caution):
The `-a` flag subscribes to every active topic on the DDS network.

Terminal
```
rosbag2 record -a -o full_system_crash_log
```

2. The Playback: `rosbag2 play`
You use this in the pit garage or the lab. Make sure your actual hardware drivers (sensors/motors) are turned off before you do this, or the bag file and the live sensors will violently fight each other by publishing to the same topics!

Standard Playback:

Terminal
```
# Replays the data at 1.0x speed
rosbag2 play lap_3_telemetry
```

The Tuning Loop (Crucial for Tech Leads):
The `-l` (loop) flag tells the bag to instantly restart when it finishes. You use this to continuously feed a 10-second cornering maneuver into your perception nodes while you tune their parameters.

Terminal
```
rosbag2 play lap_3_telemetry -l
```

3. The Visualizer: `RViz2` Configuration
You launch the visualizer in a new terminal:

Terminal
```
ros2 run rviz2 rviz2
```

How to Configure the GUI:

- **The "Add" Button:** In the bottom left, click `Add`.
- **By Topic:** Select the "By Topic" tab. This dynamically scans the DDS network (or the currently playing bag file) for visualizable streams.

- **Visualizing LiDAR:** Find your `/scan` (LaserScan) or `/pointcloud` (PointCloud2) topic and double-click it. The laser hits will appear as glowing dots in the 3D grid.
- **Visualizing Cameras:** Click `Add -> By Topic -> /camera/image_raw -> Image`. A 2D pop-up window will appear in the corner showing the video feed.
- **Visualizing the Car:** Click `Add -> By Display Type -> RobotModel`. This will read the `robot_description` parameter and render the actual 3D CAD model of your race car inside the point cloud.

3.3.3 Part 3: Best Practices & Pitfalls (CRITICAL)

Critical Debugging & Logging Pitfalls

- **The SSD Storage Nightmare (The `-x` Flag):** If your race car has two 4K cameras running at 30fps and you type `rosbag2 record -a`, you will write gigabytes of uncompressed image data to your hard drive every few seconds. You will completely fill the car's SSD mid-race, causing the OS to crash and the vehicle to stall.
  - **Best Practice:** Use regular expressions with the `-x` (exclude) flag to record everything except the massive raw images, relying on compressed images or just the LiDAR instead.
    ```
    rosbag2 record -a -x "/camera/(.*)/image_raw" -o
    safe_telemetry
    ```
- **The "Fixed Frame" Error in `RViz2`:** This is the #1 reason junior engineers think their sensors are broken. When you open `RViz2`, the "Fixed Frame" dropdown in the top-left defaults to `map`. If your car is not currently running a mapping or localization algorithm (Nav2/SLAM), the `map` frame does not exist on the network. `RViz2` will throw a massive red "Global Status: Error" and refuse to render the LiDAR data.
  - **Best Practice:** Always change the Fixed Frame to the base of your robot (e.g., `base_link`) or the odometry frame (`odom`) when doing raw sensor debugging.
- **Clock Synchronization (`--clock`):** If you are playing back a bag file to test an algorithm (like the EKF or a path planner), the algorithm will look at the messages, see a timestamp from 3 days ago, and immediately reject the data as "too old."
  - **Best Practice:** Run your nodes with `use_sim_time:=true` (from Module 3.2), and play your bag file with the `--clock` flag. This forces `rosbag2` to hijack the ROS network clock and publish the historical time as if it were the present moment.
    ```
    rosbag2 play lap_3_telemetry --clock
    ```

CHAPTER 4

Phase 4: Spatial Awareness & Kinematics

4.1 Module 4.1: TF2 (The Transform Framework)

4.1.1 Part 1: The "Why" & The "How" (Coordinate Frames & The TF Tree)

The "Why"
Sensors are selfish. A LiDAR node only knows the distance of a laser return relative to its own physical glass lens. It knows nothing about the car. If the LiDAR says "obstacle at (X: 5.0, Y: 0.0)", but the LiDAR is mounted at the very front bumper (2 meters ahead of the rear axle), the obstacle is actually 7 meters away from the vehicle's pivot point.

If we don't translate these coordinates accurately, the car will steer too early or too late and crash.

The "How" (Architecture Level)
TF2 is a distributed, time-stamped coordinate tracking system. It maintains a "Tree" of coordinate frames. Instead of doing complex 3D matrix multiplication in your head, TF2 handles the heavy lifting. You simply ask TF2: "What was the transform from the `base_link` to the `lidar_link` at exactly 0.5 seconds ago?" and it returns the precise X, Y, Z translation and Quaternion rotation.

The Standard Autonomous Racing Hierarchy:
To conform to industry standards (REP-105), you must structure your tree exactly like this:

- **`map`:** The earth-fixed, absolute track coordinate system. It never moves.
- **`odom` (Odometry):** The continuous, smooth starting point of the car. It drifts over time due to wheel slip, but it never "jumps."
- **`base_link`:** The physical center of the robot (usually the center of the rear axle for Ackermann steering).

- **Sensor Links (`lidar_link`, `camera_link`, `imu_link`):** The physical mounting points of your hardware, attached as children to the `base_link`.

4.1.2 Part 2: Real-world Autonomous Racing Use Cases

Static Broadcasters:

- **The LiDAR Mount:** The distance between the physical LiDAR sensor (`lidar_link`) and the center of the car (`base_link`) is defined by a metal bracket. It never, ever changes. We use a Static Broadcaster for this. TF2 will broadcast it once, and subscribers will cache it forever, saving massive network bandwidth.

Dynamic Broadcasters:

- **The Wheel Odometry:** As the car drives around the track, the position of the car (`base_link`) relative to its starting point (`odom`) changes 50 times a second. We use a Dynamic Broadcaster attached to the motor encoders to continuously update this mathematical relationship on the network.

4.1.3 Part 3: Implementation - The Static Broadcaster

Because static transforms never change, we rarely write C++ code for them. We launch them directly via the CLI or Python Launch files using the pre-built `tf2_ros` tools.

Via CLI (For quick track testing):

Terminal

```
# Syntax: X Y Z Yaw Pitch Roll parent_frame child_frame

# Example: LiDAR is 1.5m forward, 0.0m left, 1.0m high. No rotation.
ros2 run tf2_ros static_transform_publisher 1.5 0.0 1.0 0.0 0.0 0.0
base_link lidar_link
```

Via Python Launch File (For Production):

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='tf2_ros',
            executable='static_transform_publisher',
            name='lidar_static_tf',
            # X, Y, Z, Yaw, Pitch, Roll, Parent, Child
            arguments=['1.5', '0.0', '1.0', '0', '0', '0', '
               base_link', 'lidar_link']
        )
    ])
```

4.1.4 Part 4: Implementation - Dynamic Broadcaster and Listener

We will write a C++ and Python node that acts as our Odometry system. It will dynamically broadcast the car moving forward. Then, we will write a Listener to read that movement.

1. The Dynamic Broadcaster

Modern C++ 14/17 Implementation
File: src/odom_tf_broadcaster.cpp

```cpp
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "tf2_ros/transform_broadcaster.h"
#include "geometry_msgs/msg/transform_stamped.hpp"
#include "tf2/LinearMath/Quaternion.h" // For Euler to Quaternion conversion

using namespace std::chrono_literals;

class DynamicTFBroadcaster : public rclcpp::Node
{
public:
    DynamicTFBroadcaster() : Node("odom_tf_broadcaster"), x_pos_(0.0)
    {
        // 1. Initialize the Broadcaster
        tf_broadcaster_ = std::make_unique<tf2_ros::TransformBroadcaster>(*this);

        // 2. Broadcast at 50Hz
        timer_ = this->create_wall_timer(
            20ms, std::bind(&DynamicTFBroadcaster::broadcast_timer_callback, this));
    }

private:
    void broadcast_timer_callback()
    {
        // 3. Create the Transform Message
        geometry_msgs::msg::TransformStamped t;

        // 4. Time stamp and frame IDs are CRITICAL
        t.header.stamp = this->get_clock()->now();
        t.header.frame_id = "odom";        // Parent
        t.child_frame_id = "base_link";    // Child

        // 5. Populate Translation (Simulating moving forward on X axis)
        t.transform.translation.x = x_pos_;
        t.transform.translation.y = 0.0;
```

```
        t.transform.translation.z = 0.0;

        // 6. Populate Rotation using Quaternions (Math
           representation of 3D rotation)
        tf2::Quaternion q;
        q.setRPY(0.0, 0.0, 0.0); // Roll, Pitch, Yaw (0 radians =
            straight ahead)
        t.transform.rotation.x = q.x();
        t.transform.rotation.y = q.y();
        t.transform.rotation.z = q.z();
        t.transform.rotation.w = q.w();

        // 7. Send the transform to the TF tree
        tf_broadcaster_->sendTransform(t);

        x_pos_ += 0.01; // Move forward slightly every tick
    }

    std::unique_ptr<tf2_ros::TransformBroadcaster>
       tf_broadcaster_;
    rclcpp::TimerBase::SharedPtr timer_;
    double x_pos_;
};

int main(int argc, char** argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<DynamicTFBroadcaster>());
    rclcpp::shutdown();
    return 0;
}
```

Python Implementation

File: odom_tf_broadcaster.py

```
import math
import rclpy
from rclpy.node import Node
from tf2_ros import TransformBroadcaster
from geometry_msgs.msg import TransformStamped

def euler_to_quaternion(roll, pitch, yaw):
    # Quick math helper: ROS 2 handles rotations exclusively in
       Quaternions
    qx = math.sin(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) -
        math.cos(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)
    qy = math.cos(roll/2) * math.sin(pitch/2) * math.cos(yaw/2) +
        math.sin(roll/2) * math.cos(pitch/2) * math.sin(yaw/2)
    qz = math.cos(roll/2) * math.cos(pitch/2) * math.sin(yaw/2) -
        math.sin(roll/2) * math.sin(pitch/2) * math.cos(yaw/2)
    qw = math.cos(roll/2) * math.cos(pitch/2) * math.cos(yaw/2) +
        math.sin(roll/2) * math.sin(pitch/2) * math.sin(yaw/2)
    return [qx, qy, qz, qw]
```

```python

class DynamicTFBroadcaster(Node):
    def __init__(self):
        super().__init__('odom_tf_broadcaster')
        self.tf_broadcaster = TransformBroadcaster(self)
        self.x_pos_ = 0.0
        self.timer = self.create_timer(0.02, self.
            broadcast_timer_callback) # 50Hz

    def broadcast_timer_callback(self):
        t = TransformStamped()

        t.header.stamp = self.get_clock().now().to_msg()
        t.header.frame_id = 'odom'
        t.child_frame_id = 'base_link'

        t.transform.translation.x = self.x_pos_
        t.transform.translation.y = 0.0
        t.transform.translation.z = 0.0

        q = euler_to_quaternion(0.0, 0.0, 0.0)
        t.transform.rotation.x = q[0]
        t.transform.rotation.y = q[1]
        t.transform.rotation.z = q[2]
        t.transform.rotation.w = q[3]

        self.tf_broadcaster.sendTransform(t)
        self.x_pos_ += 0.01

def main():
    rclpy.init()
    rclpy.spin(DynamicTFBroadcaster())
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

2. The TF Listener

The Listener uses a Buffer to store the last 10 seconds of the TF tree, and queries it dynamically.

Modern C++ 14/17 Implementation Snippets

```cpp
// Includes needed:
#include "tf2_ros/transform_listener.h"
#include "tf2_ros/buffer.h"

// Inside your node's constructor:
tf_buffer_ = std::make_unique<tf2_ros::Buffer>(this->get_clock())
    ;
```

```cpp
tf_listener_ = std::make_shared<tf2_ros::TransformListener>(*
    tf_buffer_);

// In your timer loop to check where the car is:
try {
    // "Look up the transform FROM odom TO base_link at the
        latest available time"
    geometry_msgs::msg::TransformStamped t = tf_buffer_->
        lookupTransform(
        "odom", "base_link", tf2::TimePointZero);
    RCLCPP_INFO(this->get_logger(), "Car X Position: %.2f", t.
        transform.translation.x);
} catch (const tf2::TransformException & ex) {
    RCLCPP_WARN(this->get_logger(), "Could not transform: %s", ex
        .what());
}
```

Python Implementation Snippets

```python
# Imports needed:
from tf2_ros.buffer import Buffer
from tf2_ros.transform_listener import TransformListener

# Inside __init__:
self.tf_buffer = Buffer()
self.tf_listener = TransformListener(self.tf_buffer, self)

# In your timer callback:
try:
    t = self.tf_buffer.lookup_transform('odom', 'base_link',
        rclpy.time.Time())
    self.get_logger().info(f"Car X Position: {t.transform.
        translation.x}")
except Exception as e:
    self.get_logger().warning(f"Could not transform: {e}")
```

4.1.5 Part 5: CLI Tools & Debugging (CRITICAL)

When a node crashes because it "Cannot transform `lidar_link` to `map`", your engineers will be blind unless they know how to inspect the tree.

1. The Wiretap: `tf2_echo`
You need to know the exact distance and math between two frames right now.

Terminal

```
# Syntax: ros2 run tf2_ros tf2_echo <target_frame> <source_frame>

ros2 run tf2_ros tf2_echo odom base_link

# Expected Terminal Output:
# At time 1708214532.123456
# - Translation: [1.250, 0.000, 0.000]
# - Rotation: in Quaternion [0.000, 0.000, 0.000, 1.000]
```

2. The Blueprint: `view_frames` (The Most Important Tool)
If `tf2_echo` fails, it means your tree is broken. A parent is missing, or a child is detached. You need to see the entire graph visually.

Terminal

```
# Navigate to a directory where you want to save the PDF
cd ~/ros2_ws

ros2 run tf2_tools view_frames

# Expected Terminal Output:
# Listening to tf data during 5 seconds...
# Generating graph in frames.pdf...
```

> **Tech Lead Note:**
> You open `frames.pdf`. It visually maps every active coordinate frame, who its parent is, who is broadcasting it, and the average broadcast frequency. If `lidar_link` is floating alone without an arrow connecting it to `base_link`, you instantly know your static transform publisher from Part 3 crashed.

CHAPTER 5

Phase 5: Advanced Node Architecture

5.1 Module 5.1: Composition (Component Nodes)

5.1.1 Part 1: The "Why" & The "How" (IPC & Zero-Copy)

The "Why"
If you launch a Camera Driver Node and a Lane Detection Node using standard `ros2 run` commands, the OS places them in two completely separate physical processes. They cannot see each other's memory.

To send a 15-Megabyte image from the camera to the detector, ROS 2 must:

1. Copy the image from Node A's memory to the DDS middleware.
2. Serialize it into a byte stream.
3. Push it through the OS loopback network interface (Inter-Process Communication, or IPC).
4. Deserialize it.
5. Copy it into Node B's memory.

Doing this at 30 or 60 Frames Per Second will max out your CPU before your perception algorithm even begins calculating.

The "How" (Architecture Level)
Composition completely eliminates this pipeline using Intra-Process Communication (Zero-Copy).

Instead of compiling your nodes as standalone executables (`.exe` / ELF binaries), you compile them as dynamically loadable Shared Libraries (`.so`).

You then launch a single, master OS process called a **Component Container**. You dynamically load both the Camera Node and the Perception Node into this exact same container. Because they now share the same physical memory space, ROS 2 bypasses the DDS

network entirely. The Camera Node simply passes a memory pointer (`std::unique_ptr`) to the Perception Node.

Data copied: 0 bytes. Network overhead: 0 milliseconds.

Real-world Autonomous Racing Use Cases

- **The High-Speed Vision Pipeline:** You group the `v4l2_camera_driver`, the `image_rectification` node, the `YOLOv8_object_detector`, and the `track_boundary_segmentat` node into a single Component Container. The heavy 4K image data stays in one place in RAM while the different nodes take turns looking at the pointer.
- **High-Frequency Control Loops:** Grouping the Odometry filter, the Local Path Planner, and the PID Motor Controller into one container to guarantee a 100Hz+ control loop without network jitter.

5.1.2 Part 2: Implementation (C++ Component)

> **Tech Lead Note:**
> True shared-memory Composition and plugin loading is deeply tied to C++'s compilation and memory model. While Python has experimental intra-process features, production-grade Component Containers are strictly written in Modern C++ 14/17.

To make a node composable, we must change its constructor and register it as a plugin factory.

Modern C++ 14/17 Implementation

File: src/fast_perception_component.cpp

```cpp
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "sensor_msgs/msg/image.hpp"

// 1. CRITICAL: Include the macro required to register this component as a plugin
#include "rclcpp_components/register_node_macro.hpp"

// It is industry standard to wrap components in a namespace
namespace racing_perception
{

class FastPerceptionNode : public rclcpp::Node
{
public:
    // 2. The Constructor MUST take rclcpp::NodeOptions as an argument.
    // The Component Container uses this to pass arguments, parameters, and remappings dynamically.
```

```
    explicit FastPerceptionNode(const rclcpp::NodeOptions &
        options)
    : Node("fast_perception", options)
    {
        // Notice we explicitly enable intra-process
            communication in the subscriber options
        // (though the container often handles this)
        RCLCPP_INFO(this->get_logger(), "Fast Perception
            Component loaded into shared memory space!");

        sub_ = this->create_subscription<sensor_msgs::msg::Image
            >(
            "/camera/image_raw",
            10,
            std::bind(&FastPerceptionNode::image_callback, this,
                std::placeholders::_1)
        );
    }

private:
    // To achieve true Zero-Copy, we accept a Unique Pointer.
    // This proves we are taking ownership of the exact memory
        block the camera driver allocated.
    void image_callback(sensor_msgs::msg::Image::UniquePtr msg)
    {
        RCLCPP_INFO(this->get_logger(), "Processing frame at zero
            -copy memory address: %p", msg.get());
    }

    rclcpp::Subscription<sensor_msgs::msg::Image>::SharedPtr sub_
        ;
};

} // namespace racing_perception

// 3. THE MAGIC MACRO: This registers the class as a dynamically
    loadable plugin.
// Syntax: namespace::ClassName
RCLCPP_COMPONENTS_REGISTER_NODE(racing_perception::
    FastPerceptionNode)
```

The `CMakeLists.txt` Shift (Crucial)

Because we are building a library, not an executable, your `CMakeLists.txt` changes significantly:

```
# Instead of add_executable, we add a shared library
add_library(fast_perception_component SHARED src/
    fast_perception_component.cpp)

# We link the rclcpp_components library
```

```
ament_target_dependencies(fast_perception_component rclcpp
   rclcpp_components sensor_msgs)

# We register the plugin so the ROS 2 index knows it exists
rclcpp_components_register_nodes(fast_perception_component "
   racing_perception::FastPerceptionNode")
```

5.1.3 Part 3: Execution (The Component Container)

We have compiled our `.so` shared library. Now we must deploy it. This is a two-step process using the CLI.

1. Start the Master Process (The Container)
Open Terminal 1. This boots up an empty process that listens for components to be injected into it.

Terminal

```
ros2 run rclcpp_components component_container

# Expected Terminal Output:
# [INFO] [component_container]: Load Library: ...
```

2. Inject the Components dynamically
Open Terminal 2. We will now tell the container to load our newly compiled plugin.

Terminal

```
# Syntax: ros2 component load <container_name> <package_name>
<plugin_namespace::ClassName>
ros2 component load /ComponentManager my_perception_pkg
racing_perception::FastPerceptionNode

# Expected Terminal Output (in Terminal 2):
# Loaded component 1 into '/ComponentManager' node as
'/fast_perception'

# Expected Terminal Output (in Terminal 1 - The Container):
# [INFO] [fast_perception]: Fast Perception Component loaded into
shared memory space!
```

3. Inspect the Container
You can verify what is running inside the shared memory space:

Terminal

```
ros2 component list

# Expected Terminal Output:
# /ComponentManager
#     1  /fast_perception
#     2  /camera_driver
```

5.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Component Architecture Pitfalls

- **The Single-Threaded Container Freeze:** When you load multiple components into a standard `component_container`, they share the container's default Single-Threaded Executor. If your `FastPerceptionNode` executes a heavy `while()` loop or a blocking `sleep()` for 2 seconds, the Camera Driver in the same container will also freeze and drop frames. For heavy pipelines, you must launch a `component_container_mt` (Multi-Threaded container) to ensure they run concurrently in the shared memory.
- **Best Practice – The "Component-First" Architecture:** Professional teams never write standard standalone nodes anymore. Every single C++ node should be written as a Component from day one. If you later decide you want to run it as a standalone executable (for debugging or distributed networking), ROS 2 allows you to easily wrap a Component into an executable. But if you write an executable first, converting it to a Component later requires rewriting the constructor and CMake logic.
- **Unique Pointers for Zero-Copy:** If your Camera Node publishes a `SharedPtr` and your Perception Node subscribes via `SharedPtr`, ROS 2 intra-process comms will pass the pointer, but it might still execute a memory copy under the hood to preserve memory safety. To guarantee absolute zero-copy, the publisher must publish a `std::unique_ptr` (relinquishing ownership) and the subscriber must accept a `std::unique_ptr`.

5.1.5 Part 5: Automating Components with Launch Files

The "Why"
As you noted, no pit crew has the time or the terminal space to manually load shared libraries into a container during a race. The launch system must handle the memory orchestration automatically.

The "How"
We use `ComposableNodeContainer` to spawn the master process and pass it a list of `ComposableNode` objects. The Launch Executor handles the dynamic injection behind the scenes.

File: *launch/fast_perception.launch.py*

```
1 from launch import LaunchDescription
2 from launch_ros.actions import ComposableNodeContainer
```

```python
from launch_ros.descriptions import ComposableNode

def generate_launch_description():
    # 1. Define the components we want to load
    camera_component = ComposableNode(
        package='my_sensor_pkg',
        plugin='racing_sensors::CameraDriverNode',
        name='camera_driver'
    )

    perception_component = ComposableNode(
        package='my_perception_pkg',
        plugin='racing_perception::FastPerceptionNode',
        name='fast_perception'
    )

    # 2. Define the Container (Multi-Threaded for vision tasks)
    container = ComposableNodeContainer(
        name='perception_container',
        namespace='',
        package='rclcpp_components',
        executable='component_container_mt',
        composable_node_descriptions=[
            camera_component,
            perception_component
        ],
        output='screen'
    )

    return LaunchDescription([container])
```

5.2 Module 5.2: Managed Nodes (Lifecycle Nodes)

5.2.1 Part 1: The "Why" & The "How" (Deterministic Boot Sequencing)

The "Why"
Standard ROS 2 nodes boot chaotically. If your autonomous path planner boots and publishes a hard-left steering command to `/cmd_vel` *before* the steering motor driver has finished calibration, you risk hardware damage. We must enforce a strict, deterministic boot order.

The "How" (Architecture Level)
A Lifecycle Node wraps the standard node in a rigid State Machine.

1. **Unconfigured:** The node is instantiated but not connected to hardware or the network.
2. **Inactive:** Node has allocated memory and connected to hardware (e.g., opened a USB port), but it is *gagged*—not allowed to publish.
3. **Active:** Node is fully operational, publishing and acting on data.
4. **Finalized:** The node is safely shut down.

5.2.2 Part 2: Implementation (The State Machine)

We will build a `RacingMotorNode` that refuses to publish telemetry or listen to commands until it is explicitly transitioned to the `Active` state.

Modern C++ 14/17 Implementation

File: src/racing_motor_lifecycle.cpp

```
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_lifecycle/lifecycle_node.hpp"
#include "std_msgs/msg/float32.hpp"

class RacingMotorNode : public rclcpp_lifecycle::LifecycleNode
{
public:
    RacingMotorNode() : LifecycleNode("motor_controller")
    {
        RCLCPP_INFO(this->get_logger(), "Motor Node Instantiated (Unconfigured).");
    }

    using CallbackReturn = rclcpp_lifecycle::node_interfaces::LifecycleNodeInterface::CallbackReturn;

    // 1. Unconfigured -> Inactive
```

```
17      CallbackReturn on_configure(const rclcpp_lifecycle::State &)
            override
18      {
19          RCLCPP_INFO(this->get_logger(), "Configuring hardware
                ports...");
20          pub_ = this->create_publisher<std_msgs::msg::Float32>("/
                motor/rpm", 10);
21          return CallbackReturn::SUCCESS;
22      }
23
24      // 2. Inactive -> Active
25      CallbackReturn on_activate(const rclcpp_lifecycle::State &)
            override
26      {
27          RCLCPP_INFO(this->get_logger(), "Activating: Broadcasting
                enabled.");
28          pub_->on_activate(); // MUST explicitly activate
                publisher
29          timer_ = this->create_wall_timer(std::chrono::
                milliseconds(100),
30              std::bind(&RacingMotorNode::control_loop, this));
31          return CallbackReturn::SUCCESS;
32      }
33
34      // 3. Active -> Inactive
35      CallbackReturn on_deactivate(const rclcpp_lifecycle::State &)
            override
36      {
37          RCLCPP_WARN(this->get_logger(), "Deactivating: Silencing
                network.");
38          pub_->on_deactivate();
39          timer_->cancel();
40          return CallbackReturn::SUCCESS;
41      }
42
43  private:
44      void control_loop() {
45          std_msgs::msg::Float32 msg;
46          msg.data = 1500.0;
47          pub_->publish(msg);
48      }
49      std::shared_ptr<rclcpp_lifecycle::LifecyclePublisher<std_msgs
            ::msg::Float32>> pub_;
50      rclcpp::TimerBase::SharedPtr timer_;
51  };
```

Python Implementation

File: *racing_motor_lifecycle.py*

```
1  import rclpy
2  from rclpy.lifecycle import Node, State, TransitionCallbackReturn
```

```python
from std_msgs.msg import Float32

class RacingMotorNode(Node):
    def on_configure(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Configuring hardware...')
        self.pub_ = self.create_lifecycle_publisher(Float32, '/motor/rpm', 10)
        return TransitionCallbackReturn.SUCCESS

    def on_activate(self, state: State) -> TransitionCallbackReturn:
        self.get_logger().info('Activating publisher...')
        super().on_activate(state)
        self.timer_ = self.create_timer(0.1, self.control_loop)
        return TransitionCallbackReturn.SUCCESS

    def control_loop(self):
        msg = Float32(data=1500.0)
        self.pub_.publish(msg)
```

5.2.3 Part 3: Execution and Transitions (CLI Tools)

Terminal

```
# Check Current State:
ros2 lifecycle get /motor_controller

# Trigger Transitions:
ros2 lifecycle set /motor_controller configure
ros2 lifecycle set /motor_controller activate
ros2 lifecycle set /motor_controller deactivate
```

5.2.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Lifecycle Pitfalls

- **The Silent Lifecycle Publisher Trap:** If you forget to call `pub_->on_activate();` in the C++ `on_activate` callback, the node will look active, but **no data will ever reach the DDS network**.
- **Constructor vs. `on_configure`:** The constructor should be instantaneous. All heavy lifting (memory allocation, serial ports) must reside in `on_configure`.
- **Nav2 Architecture:** The entire Nav2 stack uses a master `lifecycle_manager` to orchestrate this exact sequence for professional, deterministic boots.

CHAPTER 6

Phase 6: Hardware & Simulation Integration

6.1 Module 6.1: Robot Description (URDF/Xacro)

6.1.1 Part 1: The "Why" & The "How" (URDF vs. Xacro)

The "Why"

- **URDF (Unified Robot Description Format):** An XML-based format used to describe a robot's geometry, kinematics, and dynamics. It defines **Links** (the parts) and **Joints** (the connections).
- **Xacro (XML Macros):** URDF is incredibly verbose. If you want 4 wheels on a race car, you would have to copy-paste the same 30 lines of XML for every wheel. If you then decide to change the wheel radius, you have to edit it in 4 places. This is a recipe for human error.

The Paradigm: Xacro is mandatory. It allows us to use variables (constants), math expressions, and "Macros" (functions) to generate a clean, modular URDF dynamically at launch time.

Real-world Autonomous Racing Use Cases

- **Collision vs. Visual Geometry:** For RViz, we use a high-detail `.stl` or `.dae` mesh of the car (Visual). For the Gazebo physics engine, we use a simple box or cylinder (Collision). Using a complex mesh for collision would crash the physics simulator's performance.
- **Inertial Properties:** You must define the mass and the inertia tensor (I_{xx}, I_{yy}, I_{zz}) of the battery and motors. This tells the simulator how the car will flip or slide during a high-speed turn on the track.

6.1.2 Part 2: Implementation - The XML Structure

1. The Fundamentals: Link and Joint

A **Link** is a physical piece with three tags:

- **Visual:** What we see in RViz.
- **Collision:** What the physics engine uses to detect if you hit a wall.
- **Inertial:** The mass and weight distribution (physics).

A **Joint** connects two links. Common types include `fixed` (LiDAR to Chassis), `continuous` (Wheels), and `revolute` (Steering hinges).

2. The Professional Approach: Xacro Macros

File: urdf/racecar.xacro

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" name="
   fayoum_racer">

  <xacro:property name="wheel_radius" value="0.05" />
  <xacro:property name="wheel_width" value="0.04" />
  <xacro:property name="chassis_length" value="0.4" />

  <xacro:macro name="wheel" params="prefix x_reflect y_reflect">
    <link name="${prefix}_wheel">
      <visual>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${
             wheel_width}"/>
        </geometry>
        <material name="black"><color rgba="0 0 0 1"/></material>
      </visual>
      <collision>
        <geometry>
          <cylinder radius="${wheel_radius}" length="${
             wheel_width}"/>
        </geometry>
      </collision>
    </link>

    <joint name="${prefix}_wheel_joint" type="continuous">
      <parent link="base_link"/>
      <child link="${prefix}_wheel"/>
      <origin xyz="${x_reflect * chassis_length/2} ${y_reflect *
         0.15} 0" rpy="${pi/2} 0 0"/>
      <axis xyz="0 0 1"/>
    </joint>
  </xacro:macro>

```

```
    <xacro:wheel prefix="front_left"  x_reflect="1"  y_reflect="1" />
    <xacro:wheel prefix="front_right" x_reflect="1"  y_reflect="-1" />
    <xacro:wheel prefix="rear_left"   x_reflect="-1" y_reflect="1" />
    <xacro:wheel prefix="rear_right"  x_reflect="-1" y_reflect="-1" />
</robot>
```

6.1.3 Part 3: Visualizing the Result

To see this car in RViz2, we need `robot_state_publisher`. It takes the Xacro, parses it, and broadcasts the static TF transforms (`base_link` → `wheel_link`).

Implementation: The Launch File

File: ***launch/rsp.launch.py***

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
import xacro

def generate_launch_description():
    # 1. Path to the xacro file
    pkg_path = get_package_share_directory('my_robot_description')
    xacro_file = os.path.join(pkg_path, 'urdf', 'racecar.xacro')

    # 2. Process Xacro to generate raw XML
    robot_description_config = xacro.process_file(xacro_file).toxml()

    # 3. Create the Robot State Publisher node
    node_robot_state_publisher = Node(
        package='robot_state_publisher',
        executable='robot_state_publisher',
        output='screen',
        parameters=[{'robot_description': robot_description_config}]
    )

    return LaunchDescription([node_robot_state_publisher])
```

6.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical URDF/Xacro Pitfalls

- **The "Inertial" Pitfall:** If you set a mass of 0 for any link, Gazebo will either crash or your robot will fly into the air. Every moving part must have a non-zero inertial tag.
- **The Orientation Trap:** In URDF, cylinders are created along the Z-axis. To make them look like wheels, you must rotate the visual and collision tags by $\pi/2$ (90 degrees).
- **Mesh Management:** Always use `.dae` (Collada) for visual meshes and keep file paths relative using the `package://` syntax.

6.2 Module 6.2: `ros2_control` (The Muscle System)

6.2.1 Part 1: The "Why" & The "How" (The Ecosystem)

The "Why"

If you write a custom script to talk to a specific motor driver over Serial/USB, your code is "locked" to that hardware. If you switch to a different motor driver brand, your navigation stack breaks.

`ros2_control` provides a **Hardware Abstraction Layer (HAL)**. It separates the "what" (e.g., "drive at 2.0 m/s") from the "how" (e.g., "send these specific bits over a CAN bus").

The "How" (Architecture Level)

- **Controller Manager (The Brain):** A node that loads and switches between different controllers (like an Ackermann Steer controller) at runtime.
- **Resource Manager (The Orchestrator):** The internal logic that links the Controllers to the physical Hardware.
- **Hardware Interface (The Muscle):** A C++ plugin that talks to your specific hardware (or the Gazebo simulator). It reads "Command" interfaces (outputs) and writes to "State" interfaces (inputs like encoders).

Real-world Autonomous Racing Use Cases

- **Ackermann Steering Controller:** For our team, we use a standard Ackermann controller. It handles the geometry of turning the front wheels while driving the rear ones, converting a single "Linear Velocity" and "Steering Angle" into four individual wheel speeds.
- **Seamless SITL/HITL:** You use a `mock_hardware` interface for Software-In-The-Loop (SITL) testing and swap to a `racing_car_hardware` interface for the physical race.

6.2.2 Part 2: The URDF Integration

To enable `ros2_control`, we must tell the URDF which joints are "actuated" (powered by motors).

File: ***urdf/ros2_control.xacro***

```
<ros2_control name="RealRobot" type="system">
  <hardware>
    <plugin>gazebo_ros2_control/GazeboSystem</plugin>
  </hardware>

  <joint name="front_left_steer_joint">
    <command_interface name="position">
      <param name="min">-0.5</param>
      <param name="max">0.5</param>
    </command_interface>
    <state_interface name="position"/>
  </joint>

  <joint name="rear_left_wheel_joint">
    <command_interface name="velocity">
      <param name="min">-10</param>
      <param name="max">10</param>
    </command_interface>
    <state_interface name="velocity"/>
    <state_interface name="position"/>
  </joint>
</ros2_control>

<gazebo>
  <plugin name="gazebo_ros2_control" filename="
     libgazebo_ros2_control.so">
    <parameters>$(find my_robot_controller)/config/controllers.
       yaml</parameters>
  </plugin>
</gazebo>
```

6.2.3 Part 3: The YAML Configuration

Now we define the high-level controllers that will "ride" on top of that hardware.

File: ***config/controllers.yaml***

```
controller_manager:
  ros__parameters:
    update_rate: 100  # Hz - How fast the control loop runs

    joint_state_broadcaster:
      type: joint_state_broadcaster/JointStateBroadcaster

    ackermann_steering_controller:
```

```
9         type: ackermann_steering_controller/
              AckermannSteeringController
10
11  ackermann_steering_controller:
12    ros__parameters:
13      rear_wheel_names: ['rear_left_wheel_joint', '
            rear_right_wheel_joint']
14      front_steer_names: ['front_left_steer_joint', '
            front_right_steer_joint']
15
16      wheelbase: 0.3
17      track_width: 0.2
18      wheel_radius: 0.05
19
20      publish_rate: 50.0
21      base_frame_id: base_link
```

6.2.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Control Loop Pitfalls

- **The Real-Time Loop Danger:** The Hardware Interface's `read()` and `write()` functions are executed within a **Real-Time Thread**.
  Pitfall: Adding `printf()` or `RCLCPP_INFO()` inside these functions. These are "blocking" operations. If a log takes 5ms, you have broken a 100Hz loop (10ms cycle).
  Best Practice: Only do mathematical bit-shifting or memory-mapped I/O in the hardware interface.
- **The "Ghost State" Pitfall:** If your hardware interface doesn't update the `state_interface` (encoders), controllers think the car isn't moving. They will ramp power to 100% to compensate, and the car will launch uncontrollably once it gains traction.
- **Standardized Interfaces:** Always use standard SI units: **Radians** for position and **Radians/Second** for velocity. Never use RPM or Degrees.

6.3 Module 6.3: Simulation Context (Gazebo Integration)

6.3.1 Part 1: The "Why" & The "How" (The Physics Bridge)

The "Why"
Simulation is your "Digital Proving Ground." It allows for **Software-In-The-Loop (SITL)** testing, where the exact same binaries that will run on the car's onboard computer are tested against a simulated physics engine.

- **Physics Validation:** Does the car flip when turning at 5 m/s?
- **Sensor Validation:** Does the LiDAR see the traffic cones?

- **Safety Logic:** Does the emergency brake trigger when a virtual obstacle appears?

The "How" (Architecture Level)
Gazebo is a standalone physics engine. To make it "ROS-aware," we use the `gazebo_ros2_control` plugin.

1. **Gazebo** calculates the gravity, friction, and collisions.
2. **`gazebo_ros2_control`** acts as a "Hardware Abstraction." To your ROS 2 controllers, Gazebo looks exactly like a real motor driver.
3. The plugin reads the joint commands from ROS 2, applies virtual forces to the Gazebo links, reads the resulting "simulated encoder" positions, and pushes them back to ROS 2.

6.3.2 Part 2: The Simulation URDF Tweaks (Adding Sensors)

To make the car "see" in Gazebo, we must attach sensor plugins to our URDF links.

File: ***urdf/gazebo_sensors.xacro***

```
<gazebo reference="lidar_link">
  <sensor name="lidar" type="ray">
    <pose>0 0 0 0 0 0</pose>
    <visualize>true</visualize>
    <update_rate>10</update_rate>
    <ray>
      <scan>
        <horizontal>
          <samples>360</samples>
          <min_angle>-3.14</min_angle>
          <max_angle>3.14</max_angle>
        </horizontal>
      </scan>
      <range>
        <min>0.3</min>
        <max>12.0</max>
      </range>
    </ray>
    <plugin name="gazebo_ros_laser" filename="
       libgazebo_ros_ray_sensor.so">
      <ros>
        <remapping>~/out:=scan</remapping>
      </ros>
      <output_type>sensor_msgs/LaserScan</output_type>
      <frame_name>lidar_link</frame_name>
    </plugin>
  </sensor>
</gazebo>
```

6.3.3 Part 3: The Spawn Launch File (Bringing it to Life)

This launch file orchestrates the boot sequence: starting Gazebo, spawning the entity, and loading the controllers in a deterministic order.

File: ***launch/sim.launch.py***

```
import os
from ament_index_python.packages import
    get_package_share_directory
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription,
    ExecuteProcess, RegisterEventHandler
from launch.event_handlers import OnProcessExit
from launch.launch_description_sources import
    PythonLaunchDescriptionSource
from launch_ros.actions import Node

def generate_launch_description():
    # 1. Include the Robot State Publisher
    rsp = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('my_robot_description'),
                'launch', 'rsp.launch.py'
        )]), launch_arguments={'use_sim_time': 'true'}.items()
    )

    # 2. Start Gazebo
    gazebo = IncludeLaunchDescription(
        PythonLaunchDescriptionSource([os.path.join(
            get_package_share_directory('gazebo_ros'), 'launch',
                'gazebo.launch.py'
        )]),
    )

    # 3. Spawn the robot
    spawn_entity = Node(package='gazebo_ros', executable='
        spawn_entity.py',
                        arguments=['-topic', 'robot_description',
                                   '-entity', 'fayoum_racer',
                                   '-z', '0.1'],
                        output='screen')

    # 4. Load Controllers (Delayed via Event Handlers)
    load_jsb = ExecuteProcess(
        cmd=['ros2', 'control', 'load_controller', '--set-state',
            'active', 'joint_state_broadcaster']
    )

    load_ackermann = ExecuteProcess(
        cmd=['ros2', 'control', 'load_controller', '--set-state',
            'active', 'ackermann_steering_controller']
    )
```

```
39
40      return LaunchDescription([
41          rsp, gazebo, spawn_entity,
42          RegisterEventHandler(OnProcessExit(target_action=
                spawn_entity, on_exit=[load_jsb])),
43          RegisterEventHandler(OnProcessExit(target_action=load_jsb
                , on_exit=[load_ackermann])),
44      ])
```

6.3.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical Simulation Pitfalls

- **The "Real-Time Factor" (RTF) Nightmare:** If Gazebo is lagging (RTF < 1.0), your perception algorithms will perceive the car as moving faster than it is.
  The Fix: Always run nodes with `use_sim_time:=true`.
- **The "Exploding Robot":** If wheels spawn inside the ground, physics restorative forces will launch the car into space.
  Best Practice: Always spawn the robot a few centimeters above the ground (e.g., `-z 0.1`).
- **Visualizer Performance:** `<visualize>true</visualize>` is great for debugging but heavy on the GPU. Disable it for long-duration testing to maintain a high RTF.

CHAPTER 7

Phase 7: The Autonomous Stack

7.1 Module 7.1: Sensor Fusion & State Estimation (EKF)

7.1.1 Part 1: The "Why" & The "How" (The Kalman Filter)

The "Why"
Autonomous racing is a game of millimeters. Raw data from individual sensors is often insufficient:

- **Wheel Odometry** is excellent for short distances but suffers from integration drift. If a wheel slips by just 1%, that error compounds indefinitely.
- **IMU (Inertial Measurement Unit)** provides high-frequency updates on rotation and acceleration but is inherently noisy and prone to bias.

Sensor Fusion combines these imperfect sources to produce an estimate more accurate than any single sensor could provide alone.

The "How" (The Kalman Filter)
The Extended Kalman Filter (EKF) operates in a recursive two-step loop:

1. **Predict:** Using the vehicle's last known velocity and a motion model, the EKF "guesses" the next state.
2. **Update:** The EKF compares this guess against actual sensor readings (Odometry/IMU) and calculates a **Kalman Gain**—a weighting factor that decides how much to trust the prediction versus the sensors based on their defined **Covariance** (uncertainty).

Real-world Autonomous Racing Use Cases

- **Slip Compensation:** If rear wheels lose traction during a high-G turn, the EKF sees the velocity spike but notices the IMU accelerometer does not show corresponding force. it "distrusts" the encoders to maintain a realistic pose.

- **Yaw Correction:** Wheel odometry struggles with heading. A 9-axis IMU provides absolute orientation, allowing the EKF to "snap" the heading back to reality when it drifts.

7.1.2 Part 2: The `robot_localization` Configuration

We use the `ekf_node` from the `robot_localization` package. This node tracks a 15-element state vector:

$$X = [x, y, z, \phi, \theta, \psi, \dot{x}, \dot{y}, \dot{z}, \dot{\phi}, \dot{\theta}, \dot{\psi}, \ddot{x}, \ddot{y}, \ddot{z}]$$

(Position, Orientation, Velocity, Angular Velocity, Acceleration)

File: ***config/ekf.yaml***

```yaml
ekf_filter_node:
  ros__parameters:
    frequency: 30.0
    sensor_timeout: 0.1
    two_d_mode: true  # Racing on a flat track: ignore Z, Roll, and Pitch
    print_diagnostics: true
    publish_tf: true

    # Coordinate Frames
    map_frame: map
    odom_frame: odom
    base_link_frame: base_link
    world_frame: odom

    # 1. Wheel Odometry Input
    odom0: /ackermann_steering_controller/odom
    # Matrix: [x, y, z, roll, pitch, yaw, vx, vy, vz, vroll, vpitch, vyaw, ax, ay, az]
    odom0_config: [true,  true,  false,
                   false, false, false,
                   true,  true,  false,
                   false, false, true,
                   false, false, false]
    odom0_differential: false

    # 2. IMU Input
    imu0: /imu/data
    imu0_config: [false, false, false,
                  false, false, true,
                  false, false, false,
                  false, false, true,
                  true,  false, false]
    imu0_differential: false
    imu0_relative: true
```

7.1.3 Part 3: Implementation (The Launch)

The EKF node must be integrated into the bringup stack alongside the controllers.

```python
from launch_ros.actions import Node

def generate_launch_description():
    # ... previous nodes ...

    ekf_node = Node(
        package='robot_localization',
        executable='ekf_node',
        name='ekf_filter_node',
        output='screen',
        parameters=[os.path.join(get_package_share_directory('
           my_bot'), 'config', 'ekf.yaml')],
        remappings=[('/odometry/filtered', '/odom')] # Remap for
           Nav2 standard
    )

    return LaunchDescription([
        # ...
        ekf_node
    ])
```

7.1.4 Part 4: Best Practices & Pitfalls (CRITICAL)

Critical State Estimation Pitfalls

- **The "Transform Authority" Nightmare:** A coordinate frame (`odom` → `base_link`) can only have one parent.
  The Pitfall: If the controller and the EKF both publish this transform, the visualizer will flicker violently.
  The Fix: Disable $\texttt{enable}_{odom_t}fin the steering controller and let the EKF be the sole authority.$

- **The "Non-Zero Covariance" Rule:** If a sensor reports zero standard deviation, the EKF treats it as "Infinite Truth." Any glitch can then cause a mathematical singularity (division by zero) and crash the node. Always publish a small, non-zero covariance matrix.
- **Coordinate Frame Alignment:** Ensure the IMU and Encoders agree on the forward (+X) axis. Use static transforms to align `imu_link` perfectly with `base_link` to avoid the filter thinking the car is spinning during straight-line travel.

7.2 Module 7.2: Perception & Mapping (SLAM)

7.2.1 Part 1: The "Why" & The "How" (SLAM vs. EKF)

The "Why"
You might ask: "If we have an EKF, why do we need SLAM?"

- **Local Localization (odom → base_link):** Provided by the EKF. It is high-frequency and smooth but *drifts* over time. After a 500m lap, the EKF might be off by several meters.
- **Global Localization (map → odom):** Provided by SLAM. It uses landmarks (track walls, traffic cones, pillars) to "snap" the car back to its true position on the map, effectively zeroing out the EKF drift.

The "How" (Mapping vs. Localization)

- **Mapping Mode:** The robot explores an unknown track. It uses LiDAR to identify walls, remembers their coordinates, and builds an **Occupancy Grid**—a 2D pixel map where each cell is "Free," "Occupied," or "Unknown."
- **Localization Mode:** The robot is given a pre-saved map. It compares its live LiDAR "view" to the static map. If they match, it knows its exact global coordinates (x, y, θ).

7.2.2 Part 2: Real-world Autonomous Racing Use Cases

- **The Reconnaissance Lap:** Before the race, you drive the car manually (teleop) at low speed. The `slam_toolbox` builds a high-resolution map of the track. You save this as a `.yaml` and `.pgm` file.
- **High-Speed Racing:** During the heat, you switch to "Localization Mode." The car uses the pre-saved map to stay perfectly centered on the racing line, even if wheels are slipping.

7.2.3 Part 3: The `slam_toolbox` Configuration

`slam_toolbox` is the successor to Gmapping. It is faster, more robust, and handles large-scale environments better.

File: *config/mapper_params_online_async.yaml*

```
slam_toolbox:
  ros__parameters:
    # 1. Coordinate Frames (The bridge to our EKF)
    odom_frame: odom
    map_frame: map
    base_frame: base_link
    scan_topic: /scan

    # 2. Mode: Async vs Sync
```

```
    # Async: Best for racing! Drops frames if CPU is busy rather
      than lagging.
    mode: mapping

    # 3. LiDAR Constraints
    max_laser_range: 15.0
    minimum_time_interval: 0.1
    transform_timeout: 0.2

    # 4. SLAM Parameters
    resolution: 0.05            # 5cm per pixel
    max_resample_interval: 0.5

    # Loop Closure (The "Snap" feature)
    do_loop_closing: true
    loop_search_maximum_distance: 3.0
    loop_match_minimum_chain_size: 10
```

7.2.4 Part 4: Implementation (Launch & RViz)

The Launch File Snippet

```
from launch_ros.actions import Node

def generate_launch_description():
    return LaunchDescription([
        Node(
            package='slam_toolbox',
            executable='async_slam_toolbox_node',
            name='slam_toolbox',
            output='screen',
            parameters=[os.path.join(get_package_share_directory(
                'my_bot'),
                        'config', 'mapper_params_online_async.
                            yaml'),
                        {'use_sim_time': True}] # Critical for
                            Gazebo!
        )
    ])
```

Visualizing in RViz2

1. Click **Add** → **By Topic** → `/map`.
2. Set the **Reliability Policy** to **Transient Local**. (Crucial: Maps are large and not broadcast constantly; this ensures the node receives the latest "latched" map).

7.2.5 Part 5: Best Practices & Pitfalls (CRITICAL)

Critical Mapping Pitfalls

- **The "Ghosting" Pitfall (LiDAR Vibrations):** If the LiDAR vibrates on a weak bracket, the beams wobble. SLAM perceives moving walls, resulting in a "ghost map" where walls appear doubled.
  The Fix: Use a rigid mount and increase the `minimum_travel_distance` parameter.
- **The "Loop Closure" Jump:** When the car realizes it has seen a corner before, it shifts the entire map to align.
  The Danger: A large jump (e.g., 1m) can jerk the steering of a high-speed controller. Keep the EKF tuned to ensure these jumps remain under 10cm.
- **Sync vs. Async Mode:**
  - **Sync:** Processes every scan. If CPU lags, the whole clock slows. Use only for offline, high-quality map generation.
  - **Async:** Drops scans if the CPU is overwhelmed. **Always use Async for racing.**

7.3 Module 7.3: Nav2 (The Navigation Framework)

7.3.1 Part 1: The "Why" & The "How" (The Modular Navigator)

The "Why"

A race car doesn't just need a path; it needs a strategy. It needs to know how to get from Point A to Point B (**Global Planning**), how to avoid a suddenly appearing obstacle (**Local Control**), and what to do if it gets stuck (**Recovery**).

Nav2 is the industry-standard modular framework for this. It replaces the old ROS 1 Navigation Stack with a more robust, plugin-based architecture designed specifically for production-grade robots.

The "How" (The Three Pillars)

- **Planners (Global):** Computes the long-distance path (the "Line"). It looks at the static map and finds the shortest/fastest route.
- **Controllers (Local):** Also known as the "Follower." It looks at the Global Path and generates the actual velocity and steering commands to follow that path while reacting to live sensor data.
- **Behavior Trees (BT):** The "Orchestrator." It's a logic tree that decides when to plan, when to follow, and when to trigger a "Recovery" behavior (like reversing) if the car is confused.

7.3.2 Part 2: Real-world Autonomous Racing Use Cases

- **Tuning the Inflation Layer:** Walls in a map are single lines of pixels. If a path is planned right next to those pixels, the car's physical width will cause a collision.

We use an **Inflation Layer** to create a "buffer zone." For racing, we tune this to be as tight as possible, allowing the car to "clip the apex" of a turn.

- **Ackermann Dynamics:** Unlike a differential drive robot, a race car has a minimum turning radius. We must use a controller that understands these kinematics, or Nav2 will request physically impossible maneuvers.

7.3.3 Part 3: The Nav2 Configuration (Ackermann Specific)

For racing, we use **Regulated Pure Pursuit (RPP)**. It is designed for high-speed path following and handles the non-holonomic constraints of Ackermann steering perfectly.

File: *config/nav2_params.yaml*

```yaml
controller_server:
  ros__parameters:
    controller_plugins: ["FollowPath"]

    FollowPath:
      plugin: "nav2_regulated_pure_pursuit_controller::
         RegulatedPurePursuitController"
      desired_linear_velocity: 2.0
      lookahead_dist: 0.6       # "Eyes" looking ahead on the
         path
      min_lookahead_dist: 0.3
      max_lookahead_dist: 1.0
      use_velocity_scaled_lookahead_dist: true

      # Ackermann Constraints
      min_approach_linear_velocity: 0.05
      use_approach_vel_scaling: true
      max_allowed_interpolation_dist: 0.1

planner_server:
  ros__parameters:
    planner_plugins: ["GridBased"]
    GridBased:
      plugin: "nav2_navfn_planner/NavfnPlanner" # Dijkstra/A*
```

7.3.4 Part 4: Costmap Management (Global vs. Local)

- **Global Costmap:**
  Scope: The entire track.
  Purpose: Used by the Planner to find a long-term route. Cares about static obstacles (walls).
- **Local Costmap:**
  Scope: A small "window" (e.g., 3m x 3m) around the car.
  Purpose: Used by the Controller to detect dynamic obstacles (debris, other cars).

7.3.5 Part 5: Best Practices & Pitfalls (CRITICAL)

Critical Navigation Pitfalls

- **The 'Plan vs Control' Frequency Mismatch:** Planning a 50m path is CPU intensive.
  The Pitfall: Running the Global Planner at 20Hz. If it takes 100ms to compute, the steering becomes "jerky."
  Best Practice: Run the Planner at 1-2Hz and the Controller at 20-50Hz.
- **The "Latching" Trap:** Default Behavior Trees might try to "Spin" to recover. An Ackermann car cannot spin.
  The Fix: Replace default BT recovery behaviors with "Wait" or "Replan" logic.
- **Footprint vs. Radius:** Never use `robot_radius` for a rectangular race car.
  Best Practice: Define an explicit **Footprint** (list of $[x, y]$ coordinates for the 4 corners) so Nav2 ensures no corner clips a wall.